

Part I

Computational Astrophysics

Chapter 1

What is Computational Astrophysics?

Informatica gaat net zo min over computers als astronomie over telescopen.
Informatics is as little about computers as astronomy is about telescopes.

Edsger W. Dijkstra

Computational astrophysics has come a long way since the early days when astronomers began to use support devices for their calculations. In this chapter we review some key developments in this field, embrace the relatively new concept of multiscale and multiphysics simulations, and introduce the Python programming environment which we will use for the rest of this book. We do this by presenting a series of examples which may look arcane now, but will (we hope) become clearer as we progress through the text.

1.1 Computational astrophysics

Computational astrophysics bridges observation and theory. The improved quality of observations and the need for deeper understanding of the underlying physics has driven demand for higher resolution and increased physical realism in numerical simulations. With exponentially growing hardware performance, we expect that the importance of large-scale simulations will only increase, but the software must also improve to keep pace with observations and theory.

1.1.1 Origin of this book

The idea for this book originated from writing the first version of the Astrophysical MULTipurpose Software Environment (AMUSE) in 2009, which at the time we called the MULTiphysics Software Environment (MUSE; [Portegies Zwart *et al.*, 2009](#)). Since then, we have taught numerous courses in computational astrophysics based on our software, and supervised many students in research projects based on AMUSE. The notes from these lectures and the example scripts written by our students and ourselves

formed the first version of what we called the AMUSE Primer. Later the idea gelled into a book, of which you are reading the second version right now.

Writing this book was fun. Writing the software was even more fun. Writing the example scripts was the hardest thing to do. The simplest looking codes often caused us most work. Our objective was to make these codes as short and crisp as possible. Making scripts educational without losing their main purpose was our primary challenge.

1.1.2 Hands on *is* hands on

In our experience, the best way to learn is by doing. And the best way to start doing is to take an example that resembles your objective to some degree, and build on it. We therefore include a large number of examples in a separate repository within the AMUSE code distribution, with many code snippets incorporated into this book.

In practice we prefer to work as follows:

- start with a scientific question,
- identify the minimal physics needed to address the question,
- find an example script that resembles the target problem as closely as possible (or ask a large-language model for help),
- rewrite that script to make it solve our problem.

Making figures and animations can be time consuming but also very rewarding. Making a publishable figure can take as much as a day. In our examples we adopt `matplotlib`¹ (Hunter, 2007) as our graphics library. There are many alternatives to `matplotlib` for making animations or figures, but its principal advantage is its transparent interface with Python, making it easy to incorporate into AMUSE.

A simple script can be written in a few hours; a more complicated script may take days to get right. But then the long painstaking process from test script to production code starts. This generally entails a number of stages, as illustrated in Figure 1.1. The main steps include adjusting the script to one’s needs, then refactoring the script, followed by profiling and optimization. We include the refactor step to stress the importance of preserving readability of source code, which so often leads to reproducible results (see Beck & Andres, 2004, for more about refactoring).

1.1.3 What about the math?

As Albert Einstein wrote on January 7, 1943 to nine-year-old Barbara Lee Wilson (Calaprice, 2002):

Do not worry about your difficulties in Mathematics. I can assure you mine are still greater.

¹Matplotlib is well documented and publicly available as open source from http://matplotlib.org/api/pyplot_api.html

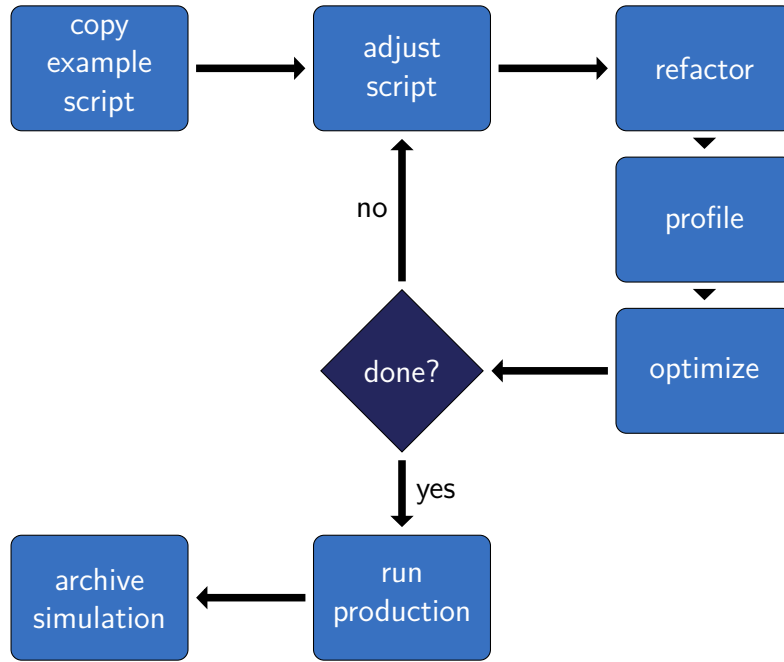


Figure 1.1: Workflow for designing a production script.

A solid background in math is nice, and surely helpful, but you can probably read and understand most of this book (up to part II) with only rudimentary mathematics and calculus. Similarly, a Ph.D. or other strong background in astrophysics or computer science will certainly make some of the material easier to digest, but we have successfully used AMUSE and this text with students at all levels, from freshman undergraduate to Masters to Ph.D. We even have users with a high-school background.

1.1.4 Alternative simulation environments

There are no real alternatives to AMUSE, at least not if you desire to perform multi-scale and multi-physics simulations. Here we mention a few projects with similar objectives that most closely comply to the listed requirements for a multi-purpose simulation environment. These include **Cactus**, a multiscale modeling framework used in **muscle3**, **FDPS**, and **MMSF**. We will briefly discuss each of these frameworks.

Cactus (Goodale *et al.*, 2003) is a general-purpose problem-solving environment for the physical sciences (Allen *et al.*, 2001). It is modular and supports parallel computing across the network with a variety of hardware. **Cactus** was last updated (version 4.12 of May 2022).² **Cactus** offers several advantages over other frameworks. It uses a component-based architecture, where the core functionality is provided by a central “*flesh*” and various “*thorns*” (modules) that can be independently developed, compiled, and loaded. The thorns can be parallelized, for multi-core architectures and distributed computing environments. Its high level of abstraction and interoperability with other tools simplifies the development of complex simulations. Using **Cactus** can be very rewarding, but understanding the details is cumbersome, and it may require

²see <http://www.cactuscode.org>

numerous recompilations to generate a working implementation.

FDPS stands for Framework for Developing Particle Simulators (Iwasawa *et al.*, 2016)³. It is built with much the same philosophy as AMUSE, but is much more coarse-grained. If FDPS were some sort of Lego (in software terms) for simulating self-gravitating systems, then AMUSE would be more like Duplo: big blocks that allow you to build an interesting colorful building with only a little effort.

FDPS is a high-performance, open-source library, written in C++ and designed to simplify and accelerate the development of large-scale, parallel particle-based simulation codes—such as N -body or SPH simulations—on distributed-memory supercomputers (Iwasawa *et al.*, 2016). *Petar* (Wang *et al.*, 2020), the N -body code used in Torch (Section 10.2.1), is written using FDPS. It provides a good example of how code written in FDPS can be embedded in AMUSE.

FDPS is optimized for building fast, scalable, single-physics particle simulation codes from scratch, with a focus on parallel performance, whereas AMUSE is designed for integrating and orchestrating multiple existing codes within a high-level, user-friendly Python environment, prioritizing flexibility and multi-physics capabilities. Researchers should choose between them based on whether their priority is raw computational performance for custom particle simulations (FDPS) or flexible, multi-physics coupling and ease of scripting (AMUSE). As with *Cactus*, FTSP also requires recompilation after changes are made.

Finally, we briefly mention the Multiscale Modeling and Simulation Framework (MMSF, Borgdorff *et al.*, 2013; Chopard *et al.*, 2014), which is based on a similar philosophy as that underlying AMUSE. MMSF systematically organizes and couples models at different scales, providing a rigorous and flexible approach to simulating complex, multi-scale systems. It is modular and is supported by formal languages and software tools for efficient implementation and execution. MMSF has been applied effectively in the *muscle3* environment (Veen & Hoekstra, 2020). However, in contrast to AMUSE’s structure of a driver script instructing a set of models to run and exchange information, it instead manages a set of distributed, mostly independent models which exchange messages via externally configurable connections, which block if a result they need is not yet available.

All three framework environments require recompilation before reuse, and are therefore not as suitable for experimentation as AMUSE. Each of them, however, is easier to tune to the appropriate hardware environment, but may lack native GPU support. Each package has its advantage and disadvantages. AMUSE stands out for its ease of use, breadth of included solvers, robust unit and data handling, and its focus on reproducible, modular, and flexible multi-physics simulations. AMUSE and FDPS are geared towards astrophysics, whereas MMSF, *Cactus*, and targets a broader range of applications.

1.1.5 Objective of this book

This book is about how to perform simulations of astrophysical phenomena. These simulations have the goal of mimicking nature, or at least certain aspects of it, in

³see: <https://github.com/FDPS/FDPS>.

order to better understand the universe. By the end of this book you will be able to perform many high-end simulations yourself. You will have learned how to set up initial conditions, how to design and develop a simulation environment to address your astrophysical question, how to perform the runs, and how to analyze the resulting data.

You can read this book in several ways. You can simply start at the beginning and work through to the end, or you may prefer to pick out the pieces you need in order to bootstrap your own research project. Either way, we recommend that you read the first couple of chapters, to understand how AMUSE works and some of its basic programming ideas.

1.1.6 What is missing from this book

This book is far from exhaustive. Many topics, such as data mining, high-performance and distributed computing, programming paradigms for graphics processing units, operating systems, and the effects of limited numerical precision are entirely separate disciplines that we can only direct you to. AMUSE contains far more functionality than we can describe. We will only briefly address the algorithms underlying many of the phenomena discussed. Our discussion of the underlying physics is also necessarily limited. This is not an astrophysics text—rather, our goal is to present just enough astrophysics to allow you to harness the modules we describe.

Of the many astrophysical phenomena we could include, we have decided to limit our scope to gravitational dynamics, stellar evolution, hydrodynamics, radiative transfer, and chemistry. We neglect a lot of physics, including asteroseismology, astrobiology, planetary geology, cosmology, YORP⁴, and tidal dissipative effects. These are not unimportant, but the book is finite, and new modules addressing such areas are constantly being added to the AMUSE framework.

1.1.7 Outline of the book

This book is divided into three parts. In Part I, Chapters 4 to 7, we discuss scripts designed to address single physical processes, for which only one numerical solver is required. A *minimal solver* is the simplest possible script needed to run a particular solver. In AMUSE, the low-level solvers—usually written in high-performance languages and often optimized for parallel architectures—are called *community codes*. When we include the AMUSE Python interface structure we generally use the more generic term *community module*. In practice, we often need additional modules for generating initial conditions, boundary values, or post-production analysis, but the main work in a minimal code is performed by a single community module. Minimal solvers run a particular code and stop after producing just enough information to gauge the quality of the solution. The solvers presented here should be viewed as examples to illustrate the AMUSE programming style and module interfaces, but they can also be used as a basis for further work.

⁴The Yarkovsky-O’Keefe-Radzievskii-Paddack effect (Rubincam, 2000) influences the spin of a minor body by reprocessing stellar radiation as thermal emission.

The next level of complexity is a *simple solver*, which is an extended version of a minimal solver. Simple solvers are meant to illustrate more complex and practical mono-physics applications. Within the AMUSE framework we can run a single code as a standalone program. Since AMUSE contains multiple solvers for each physical domain it addresses, this allows for detailed and instructive comparison between them. The ease with which initial conditions can be generated, and with which the user can change solvers and analyze the results makes running multiple solvers on the same problem a useful exercise. In this regard, the main advantage of using AMUSE is the unified way in which the input is parsed and delivered to, and results returned from, the solver. This enables the possibility of using exactly the same initial conditions for each of a set of numerical solvers, and analyzing the resulting data in a uniform way.

In Part III, Chapters 8 and 9, we discuss elementary and hierarchical code coupling strategies, followed by Chapter 10, in which we present a few example cases for multi-scale and multi-physics simulations.

In the last part of the book, Part III, we dwell on expert knowledge and discuss the subtleties of multi-scale and multi-physics simulations. In the last chapter (Chapter 10), we discuss possible expansions and other packages derived from AMUSE.

The chapters are followed by several appendices. In Appendix A we further discuss the fundamental concepts of AMUSE, and give details about its internal operation. Important, but often not considered multi-scale or multi-physics are the generation of initial conditions, which we discuss in Chapter 3. How to add your own code to AMUSE is discussed in Appendix B. The last appendix includes a brief Python primer (Appendix C).

We have adopted this presentation order for its steady increase in physical complexity and its systematic introduction of more and more sophisticated AMUSE capabilities.

1.2 A brief history of simulations in astrophysics

The first recordings of theoretical astronomy date back to the Egyptian 18th dynastic (1550–1292 BCE) calculations by pharaoh Hatshepsut’s architect Senmut; portions of the murals found in his tomb at Deir el-Bahri can be found at <http://www.metmuseum.org> (Dorman, 2006). Figure 1.2 presents part of this mural. Such calculations enabled astronomers to recognize structure and describe patterns in the heavens, an aspect of astronomy that is still relevant today.

Although the Egyptian civilization did not show much academic interest in the heavens, this changed dramatically when the Babylonian civilization emerged. Around 350 to 50 BCE, they already showed a keen interest in mathematics and algorithms. This is demonstrated by in several clay tablets with cuneiform writing, such as the one presented in Figure 1.3, which shows a calculation of Jupiter’s orbital integration on the sky. The complexity of this calculation, due to the epicycles the planet makes when its orbit is folded in with the Earth’s and subsequently projected on the sky, requires a second order integration. The method depicted in Figure 1.3 resembles a Leap-frog or Verlet integrator. Although we do not know who wrote the tablet, or who designed the algorithm, it is clear that the Babylonians were already aware of 2nd

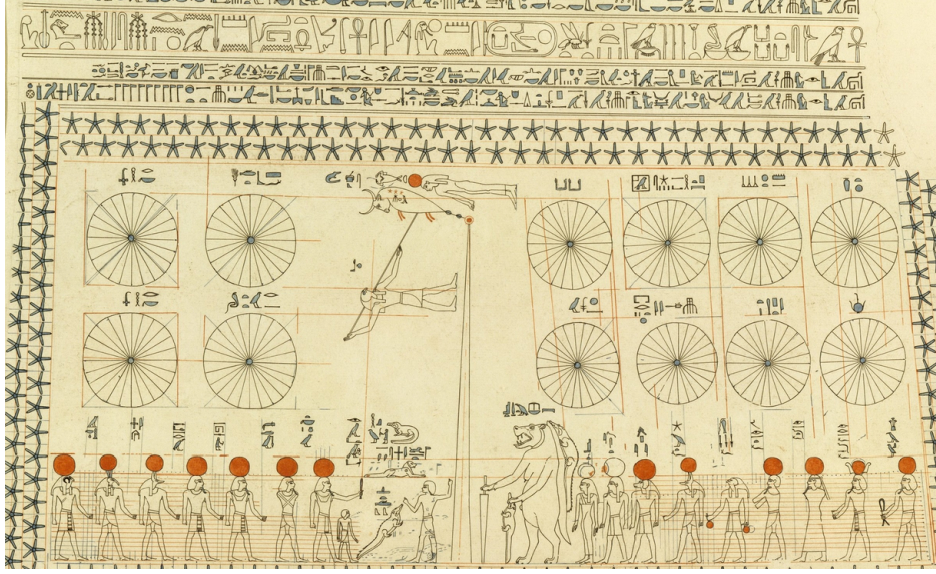


Figure 1.2: Reproduction of the lower part of the 2nd tomb of Senenmut’s stela, dated around 1479-1458 BCE (accession number MMT:48.105.52 (TT353), Figure 27 of [Wilkinson, 1979](#), GNU General Public License, Rogers Fund, 1948). This unique mural shows, apart from all the Egyptian gods, the most important constellations and a much debated conjunction between the Sun and the Pleiades star cluster. The original photograph was presented in [Dorman \(2006, 1991\)](#).

order error-reducing algorithms ([Ossendrijver, 2016](#)). It would be better to call this the Babylonian integrator.

Many centuries later, modern astronomy started with the discovery of the first refracting telescope, built by Hans Lippershey in 1608, which had a collecting area of about 1 cm^2 . Galileo Galilei reported the first astronomical applications in 1609, using a telescope of similar design. In the 400 years since that momentous first application of new technology, the collecting areas of the largest research telescopes have increased by a factor of about 10^6 , becoming the 10-m class telescopes in widespread use today.

Digital computers were introduced in the late 1930s ([Zuse, 1967, 1993](#)), and the first commercial programmable model, ENIAC, achieved in 1945 a computational speed equivalent to about 200 modern floating point (double precision) operations per second (flops). (We say “equivalent” because ENIAC was essentially a single precision machine.) Today the fastest supercomputers achieve speeds exceeding 10^{18} (Exa) flops, a speedup of a factor of roughly 10^{15} in raw performance in just eight decades. Thus, while observing technology has grown impressively but relatively slowly over the past four centuries, computer scientists have witnessed a technological explosion in living memory.

The ongoing revolution in the availability of digital computers has led to an entirely new branch of research in which key facilities are not confined to high mountain tops in Hawaii or the Chilean Andes, but more often in the room next door. Astronomers quickly realized that they could use computers to archive, process, analyze, and mine the copious amounts of data taken by observing campaigns. However, arguably the biggest impact on the way astronomers pursue their scientific questions has been in numerical simulations of astrophysical phenomena.



Figure 1.3: Photo of text A (lines 1 to 7) of the Babylonian clay tablet from 350 to 50 BCE with the mathematical tables to predict Jupiter's orbit on the sky. The original photograph was presented as figure 1 in [Ossendrijver \(2016\)](#). The tablet is about 4×4 cm.

Astronomical research lends itself naturally to computation because it is characterized by large ranges in temporal and spatial scales, complex and coupled nonlinear processes, and the extreme conditions found in interstellar space. With the appearance of digital computers it became possible to study processes in the intergalactic vacuum, hot plasmas at stellar surfaces, billion-year time scale processes, and black-hole physics, none of which can be studied in Earth-based laboratories. The major current limitation to studying the universe by means of computation lies in software, and in particular our limited understanding of algorithms that can solve multiscale, multiphysics problems. AMUSE is an attempt to address these issues.

Much help in generating AMUSE scripts can be expected from artificial intelligence via Large Language Models (LLM). One can simply ask a LLM to write you a script using AMUSE, and since these models are trained on the book's first version examples, it generally does a reasonable job. They tend to be a bit confused about channels and bridges, though, which might indicate that our explanations are not always as easily apprehended by an AI.

1.2.1 The first simulation experiments

One of the first astrophysical simulations was carried out in 1941 by Erik Holmberg, a graduate student at Lund University who had access to a number of photocells.

Using the fact that the intensity of radiation drops off as the inverse square of the distance, like the force of gravity, he devised an experiment to simulate two colliding galaxies. The initial setup for a single “galaxy” in Holmberg’s experiment is presented in Figure 1.4. The progress of his experiment is shown in Figure 1.5. The arrows indicate the directions in which the galaxies move with respect to one another, and how they rotate. This experiment made Holmberg one of the founders of what is now an immense body of astronomical research in gravitational dynamics, some of it reviewed by [Binney & Tremaine \(1987\)](#).

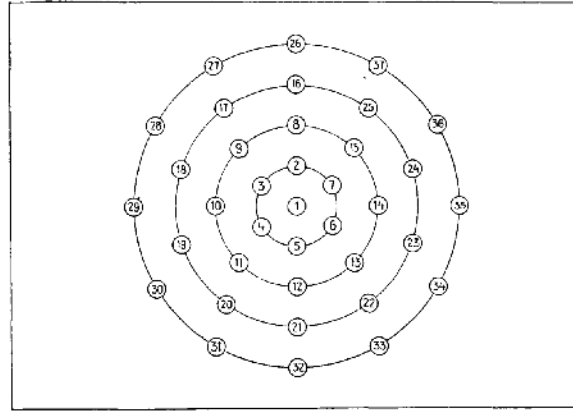


Figure 1.4: Copy of Figure 3 from [Holmberg \(1941\)](#), presenting the initial conditions for a single galaxy in Holmberg’s simulation of two colliding galaxies. (Copyright AAS. Reproduced with permission.)

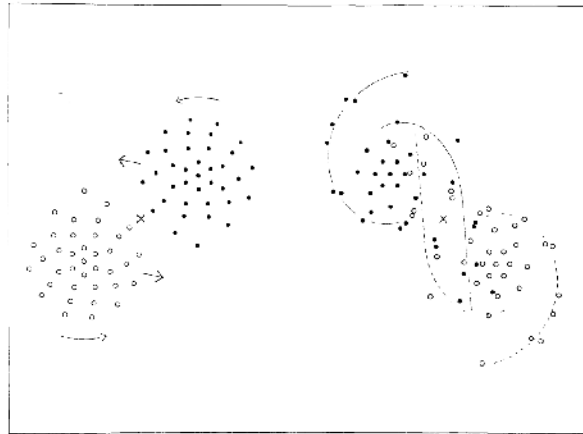


Figure 1.5: Copy of Figure 4a from [Holmberg \(1941\)](#), presenting two snapshots of the results from his calculations. Both snapshots were taken at the moment the two galaxies were separated by twice their radii, one before the interaction (left) and one after the first close approach (right). (Copyright AAS. Reproduced with permission.)

Holmberg had no access to a computer, so all of his calculations were performed by hand. His article ([Holmberg, 1941](#)) is interesting in many respects, but unclear on exactly how the forces (photon counts in his case) were used to update the positions of the particles (the light bulbs). Today, in a comparable paper, we might state that the integration was performed using (say) a 4th-order Hermite predictor-corrector scheme

with direct force evaluation and zero softening (see Makino & Aarseth, 1992). The choice of algorithm (in this case the Hermite scheme) is an important part of the solution. Although this level of detail might sound excessive, you will better appreciate such considerations by the time you have read Chapter 4.

1.3 Software used in this book

1.3.1 Motivation for a homogeneous software environment

Developing high-quality simulation software is relatively straightforward when the physical scope of the problem is limited. The task becomes considerably more difficult once the application spans multiple physical domains, or if the dynamic range in temporal or spatial scales exceeds the precision of the hardware. Successful examples of complex multiphysics software in computational astrophysics are NBODY7 (Aarseth, 1999, 2011), FLASH (Fryxell *et al.*, 2000, 2010), and GADGET (Springel *et al.*, 2001; Springel, 2000), which have broad user bases and can be applied to a wide variety of problems. (Note that, where possible, we provide two references for each software implementation, one to the publication in which the package was first described, and one to the Astrophysics Source Code Library at <http://ascl.net/> or ZENODO <https://zenodo.org/>) Some of these codes address relatively narrow ranges of physics and are limited by the specific numerical algorithms or data topology adopted to solve the underlying equations, but others, such as FLASH, are highly modular in their internal organization, in some ways similar to AMUSE. Most of these software is hosted in US servers. In the near future we expect to move all our activities to a less autocratic location.

Figure 1.6 presents typical temporal and spatial scales for a number of astronomical phenomena. The ellipses indicate specific areas of interest. The colors indicate physical domains: gravity (blue), stellar evolution (red), hydrodynamics (yellow), and radiative processes (green). Several domains are identified by name, such as the collapse of a giant molecular cloud (top right in yellow), and the range of habitable planet orbits (blue circle at lower left). Gravity and hydrodynamics span a broad range of temporal and spatial scales, whereas radiative processes and stellar evolution exhibit much narrower ranges. Note that both spatial and temporal scales span a range of about 14 orders of magnitude, from galactic orbits at one extreme to stellar mergers or intricate orbital dynamics at the other.

Many existing simulation codes provide excellent solvers within their specific physical domains, but astrophysical problems often require solvers for a wide variety of domains. For example, understanding the ejection of primordial gas from an embedded star cluster, born during a galaxy merger, requires simultaneous treatments of collisional stellar dynamics (the protostellar cluster), collisionless stellar dynamics (the galaxy merger), hydrodynamics (interstellar gas), stellar evolution (embedded protostars and stars), and radiative processes (feedback between stars and the interstellar gas). All these processes occur on comparable temporal and spatial scales.

Instead of designing new algorithms to solve all of these problems simultaneously, we adopt the philosophy that each of the physical domains can be solved separately, then

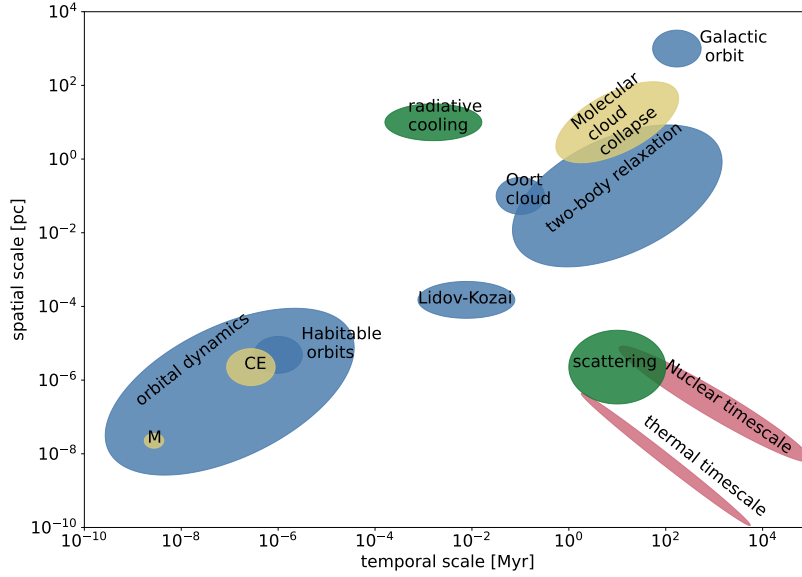


Figure 1.6: A number of typical scales, for gravitational dynamics (blue), stellar evolution (red), hydrodynamics (yellow), and radiative transfer (green), are indicated at several locations in the plane of temporal and spatial scales. Here several small scales (mergers indicated by M) and common envelope evolution (indicated by CE) are embedded within the orbital dynamics scale ellipse.

combined into a consistent solution at discrete intervals in time and space. Validation of such a simulation could subsequently be accomplished by recomputing the model with a different solver for each of the physical domains. In this way we can study the convergence (or the absence thereof) of the solution with a sequence of numerical implementations. The Astrophysical Multipurpose Software Environment (AMUSE; Portegies Zwart, 2011; Portegies Zwart *et al.*, 2013) allows a user to combine existing solvers to build new applications that can be combined again to study increasingly complex problems. This enables the growth of multiphysics and multiscale application software in a hierarchical fashion, and allows testing of each intermediate step as the complexity of the software increases.

AMUSE has a hierarchical component architecture that encapsulates dynamically shared libraries for simulating stellar evolution, gravitational dynamics, (magneto)-hydrodynamics, radiative processes, and also has limited support for astrochemistry. Applications are able to use some or all of these resources simultaneously, and can exchange data in a unique and deterministic way. With this hybrid simulation environment it becomes possible to study multiphysics processes operating on a broad range of length and time scales.

1.3.2 Choice of programming languages

We have adopted Python (van Rossum, 1995a,b,c,d) as the implementation language for the AMUSE framework and high-level management functions, including bindings

to the communication interface. The earlier version of this book used Python2.7, but we have since migrated to Python3 [Van Rossum & Drake \(2009\)](#) (as of writing 3.8 or higher)⁵. The choice of Python is motivated by its flexibility, broad acceptance in the scientific community, object oriented design, and ability to allow rapid prototyping, which shortens the software development cycle and enables easy access to the community code in the community module, albeit at the cost of reduced performance ([Portegies Zwart, 2020](#)). However, the entire AMUSE framework is organized in such a way that relatively little computer time is actually spent in the framework itself—most time is (or should be) spent in the community code(s). The overhead from Python compared to a compiled high-level language is typically $\lesssim 10\%$, and often much less ([Pelupessy et al., 2013](#)).

As discussed further in Paragraph [B.1.1.2](#), our implementation of the communication layer in AMUSE uses the standard Message Passing Interface protocol (MPI; [Gropp et al., 1996](#)) and SmartSockets ([Maassen & Bal, 2007](#)). Normally MPI and SmartSockets are used for communication between compute nodes on parallel distributed-memory systems or within computational grids. However, in AMUSE they are used as communication channels between *all* processes, whether or not they reside on the same node as the control script, and whether or not they are themselves parallel.

AMUSE community codes are written in a variety of languages, reflecting the programming backgrounds and preferences of their authors. As a result, for each community code there are two interface functions. The one on the AMUSE side is written in Python and is called the proxy. The interface on the community code side is called the partner and is written in the native language of that code. These two interfaces communicate via one of the message-passing protocols just described. Thus our only real restriction on supported languages is the requirement that the community code is written in a language with MPI or Smart Sockets bindings, such as C ([Ritchie, 1993b,a](#)), C++, ([Stroustrup, 2013](#)), Java ([Gosling et al., 1996, 2014](#)), and members of the FORTRAN family ([Backus, 1978](#)).

We emphasize that, from the user perspective, the details of the interface and communication protocols are invisible. These details are central to the AMUSE design, but most high-level users are blissfully unaware of them. To the user, a Python AMUSE script looks just like a traditional serial program.

1.3.3 Design of AMUSE

The AMUSE project goal stipulated three basic functionalities:

- (1) a homogeneous, physically motivated interface for existing astronomical simulation codes;
- (2) the incorporation of multiple community codes from four fundamental domains (stellar evolution, gravitational dynamics, hydrodynamics, and radiative transfer); and

⁵Python version 3.13 was the most recent release while writing this.

- (3) the ability to design new simulation experiments by combining one or more of the community codes in various ways.

In this way researchers can concentrate on physically relevant details, such as the calling sequence for the various physical domains, without having to worry about the many technical details.

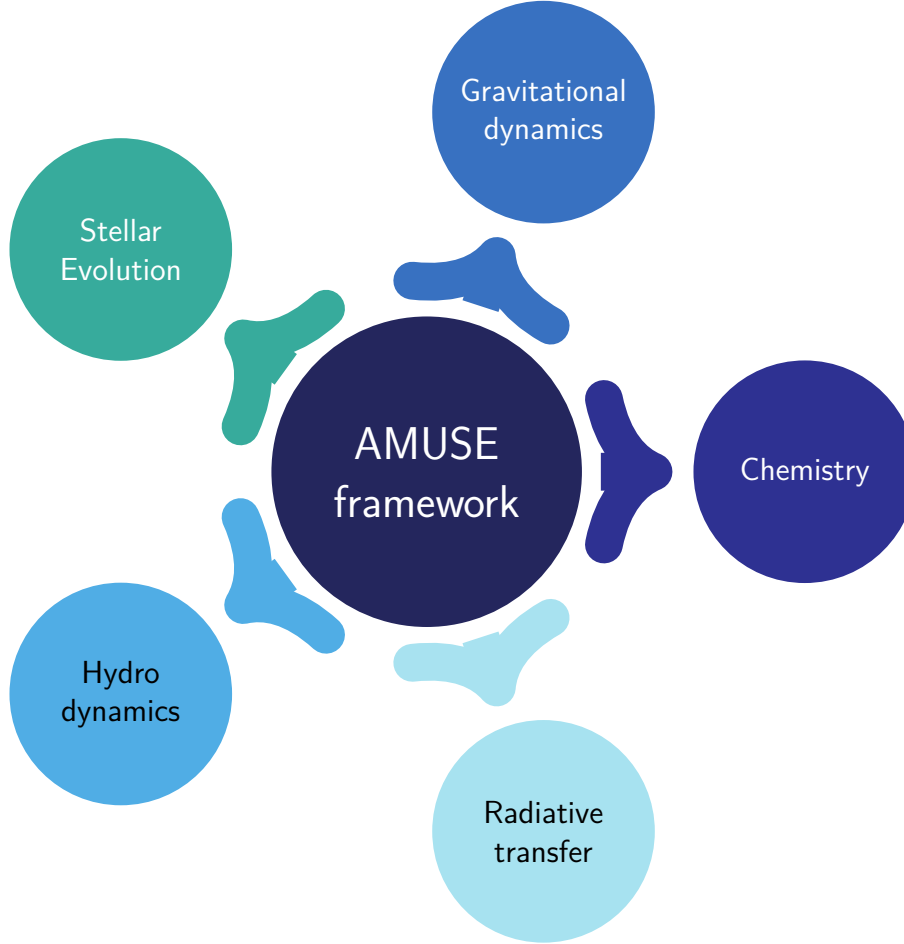


Figure 1.7: Multiscale and multiphysics codes are connected disciplines, as depicted here in the domain–application plane.

Each of the existing community codes in AMUSE is embedded in an interface layer that realizes communication between the AMUSE framework and a community code. One important advantage of this strict separation is that it explicitly prohibits short-cut communication between independent subroutines, greatly diminishing the opportunities to create “spaghetti code”⁶. In addition, it allows us to manipulate data as it moves between the community code and the framework. One such data manipulation is the conversion of units, as discussed in Section 3.4.1. The handling of community-code dependent parameters are largely managed by the framework, as is the acquisition and mining of simulation data. Figure 1.7 illustrates schematically

⁶In our opinions the term “ramen code” would be more appropriate.

the philosophy of AMUSE, with the central AMUSE framework surrounded by its four fundamental physical solvers.

New hybrid codes can be written in AMUSE by combining community codes using Python. An AMUSE Python script for solving a particular problem prescribes the order in which events are handled and specifies the numerical solver that should be used to resolve each event. Once initial conditions are generated and the simulation is completed, the resulting data can be stored or further analyzed using the tools available in AMUSE, or developed by the user. The technical problems associated with coupling codes, setting boundary conditions, specifying run-time parameters, and guaranteeing that the various elements communicate in the proper units are handled entirely by AMUSE. A thorough understanding of the limitations of each of the incorporated codes is important for assessing the quality of the simulation, but the burden of managing the technical details is not forced on the user.

We have demonstrated modularity and interchangeability of the AMUSE community codes for solving the basic physical processes described above. The user can select from a variety of gravitational N -body solvers (symplectic N -body codes, direct N -body codes, and tree codes) by changing a single word in the script. The same holds for stellar evolution codes (parameterized and Henyey codes), hydrodynamics solvers (smooth particle hydrodynamics and adaptive mesh refinement codes), and radiative transfer codes (Monte Carlo photon transport codes, and tessellated photon flow solvers). The modularity, and how a complex cascade of numerical solvers can grow to an end product, is illustrated in Chapter 10, and in Part III. This cascade of jargon may sound like Jabberwocky (Carol, 1871) right now, but we hope that much of it will eventually make sense by the time you reach the end of this book.

1.3.4 Terminology

The basic structure of the AMUSE framework is illustrated in Figure 1.8. The figure is subdivided into three blocks. The top block is mostly user written, the middle block implements the interfaces onto the community codes, and the bottom block is the community modules and the actual community source code, which is connected to the rest of the framework via MPI or sockets.

The components can be described as follows:

- The Python *user script* (top layer in Figure 1.8), which addresses a specific physical problem or set of problems, in the form of system-provided or user-written Python commands that serve as the user interface onto the framework. This is the only part of a production script that needs to be written by the user. The coupling between community codes is implemented in this layer, with the help of support classes for converting units and data structures provided by AMUSE (see Section 2.3.1).
- The *community module*, comprising three key elements:
 - The *manager* (second layer in top block), which provides an object-oriented interface onto the communication layer via a suite of system-provided utility

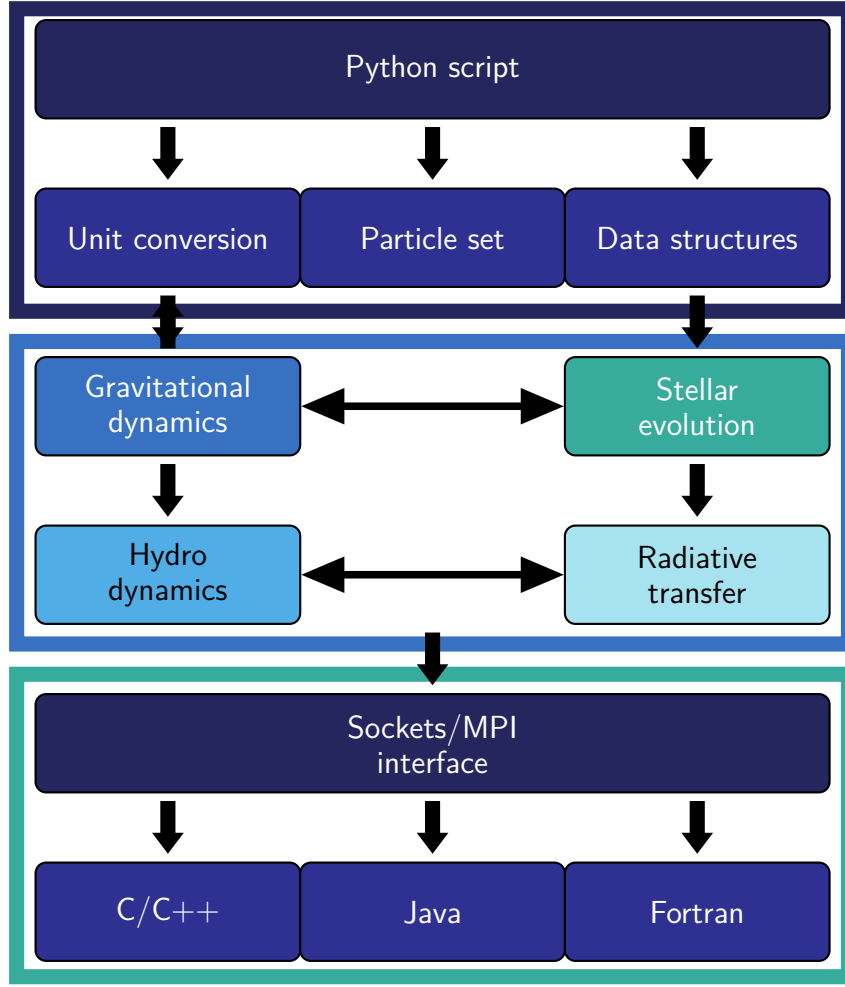


Figure 1.8: General structure of the AMUSE framework. The user writes the Python script (top layer). The lower layers in the first block are responsible for units and data structures. The framework in the second block represents the interfaces to the community modules (dark blue for gravitational dynamics, green for stellar evolution, funny blue for hydrodynamics and radiative transfer in light blue to conform with the color coding of Figure 1.7), which connect via sockets or MPI (third block) to the community codes at bottom. Information flows downward to the community modules, as indicated by the arrows, as well as upward, back to the Python script.

functions. This layer handles unit conversion, and also contains the state engine and the associated data repository, all of which are required to guarantee consistency of data across modules. This layer is generic—it is not specific to any particular problem or physical domain.

- The *communication layer* (second block), which realizes the bidirectional communication between the manager and the community code layer. This is implemented via a proxy and an associated partner, which together provide the connection to each community code.
- The *community code layer* (third block), which contains the actual community codes and implements control and data management operations on them. Each piece of code in this layer is domain-specific, although the code may be designed to be very general within its particular physical domain.

1.4 Installing AMUSE

Before you continue reading, we advise you to install the AMUSE package. Here is a brief overview of the installation steps. We assume that you are working in a Linux or macOS environment. We don't recommend or support AMUSE under Windows, except within a Linux environment such as Windows subsystem for Linux⁷ or Multipass⁸. For the following we assume that you have a basic understanding of your operating system and the command line.

To install AMUSE, we strongly recommend creating a new virtual environment, either via `conda`⁹ or via Python3's `venv`¹⁰. AMUSE can be installed either as a package or from source in either. Other virtual environments could work equally well, but are not supported by us at the time of writing.

1.4.1 Installing AMUSE in a virtual environment

Here we will start from a `python venv` or `conda` environment on a Ubuntu linux laptop. This tutorial was written on a fresh Ubuntu 24.04.2 LTS (6.5.0-1027-oem) installation on a Lenovo P14s laptop. This is not a particularly high-end computer, but it should suffice for all example calculations in this book. If only, because all the example calculations in this book are generated with this same machine. We would like to emphasize here that the revenues we receive for writing this book is by far insufficient to purchase such a laptop.

Open a terminal and copy the lines below to create an environment for your project (note that the leading characters only indicates the terminal prompt, and it may be different on your computer. The instructions are written for `bash` or another Bourne shell.

```
%> git clone git@github.com:amusecode/amuse.git .  
%> cd amuse  
%> ./setup
```

This downloads the AMUSE source code and runs the AMUSE setup script. This script will detect the environment you are using and guide you through the installation process.

Since the details of the installation process may evolve over time, we do not print the full instructions here.

However, if you feel uncertain about what to do, try:

```
%> ./setup help
```

which will give you some options.

After following the instructions, the AMUSE framework should be built, and probably most of the packages and libraries.

⁷https://en.wikipedia.org/wiki/Windows_Subsystem_for_Linux

⁸<https://multipass.run>

⁹<https://conda-forge.org/download>

¹⁰<https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments>

In case something is broken, you can visit our GitHub issue tracker (<https://github.com/amusecode/amuse/issues>) and ask for help.

1.4.1.1 Testing your AMUSE installation

After a successful installation, we encourage you to run a standard set of tests. The general tests can be run quite similarly to installing using the setup routine

```
%> ./setup test amuse-framework
```

for testing the framework, or

```
%> ./setup test ph4
```

testing an individual code, such as ph4 in this case.

1.4.2 Running the examples

Examples in this book, and throughout the entire AMUSE framework can be run by invoking the python `examples` package

```
%> ./python -m amuse.examples.{example_name}
```

Here `example_name` is the name of the example.

To make this work, one first has to install the amuse example package through pip.

```
%> pip install amuse-examples
```

And then run

```
%> ./python -m amuse.examples.sun_venus_earth
```

will run the example script `sun_venus_earth` and reproduce Figure 3.4.

1.4.3 Running AMUSE

By now, you should have built and tested AMUSE, as described in Section 1.4, and are probably wondering “What can AMUSE do for me?” This section will get you started. If you’re new to Python you’ll find the official Python documentation¹¹ a valuable resource. Most of the scripts in this book were rewritten with at least Python version 3.7 in mind, but much of the coding will remain up-to-date with the current version 3.12 and higher.

You can run AMUSE interactively from Python or iPython:

```
%> python
```

Once Python has started, you can load AMUSE by

```
from amuse.lab import *
```

¹¹<https://docs.python.org/3>

This line is a lazy but very quick way to import most of the AMUSE package in a single call. However, we generally do not advise it because it pollutes the Python name space. Rather, we recommend that modules are imported from AMUSE individually, and in the examples that follow this is how we will present source code.

The code examples and snippets presented in the book are designed to be cut and pasted into your Python interpreter or your file editor. However, due to limitations of online book formatting, whitespace may not be conserved when doing so. To facilitate using the code examples directly, we provide an `amuse-examples` package that contains the source code for all examples in this book as well as all the generated figures. Once installed the examples can be imported in the following way (using the `ae` namespace as shorthand for `amuse.examples`):

```
import amuse.examples as ae
```

The complete source code of an example (e.g. `gravity_minimal`) can be seen with the `ae.print_example` function:

```
ae.show("gravity_minimal")
```

If you are using AMUSE from a Jupyter notebook, you can copy the example directly to a new cell, ready for editing:

```
ae.to_cell("gravity_minimal")
```

To run an example with default settings, you can use the `run` function:

```
ae.run("gravity_minimal")
```

We hope that these examples and code snippets encourage you to experiment and initiate interesting research projects.

1.4.3.1 Running AMUSE without network access

Running AMUSE without a network connection is curiously not always supported by your favorite operating system. The underlying reason hides in the requirement for MPI to have a network connection. This can be solved by using the smart sockets connection channel, or by using a loopback network. The latter can be done with the following command line:

```
%> mpirun -n 1 --mca btl_tcp_if_include lo python -m amuse.examples.gravity_minimal
```

It's a rather complex command, but fundamentally, it boils down to running the MPI command `mpirun` using a single core with the specific command for process binding for the tcp protocol on your local network. This so-called `MCA` command (`-mca`) can be invoked in `mpirun` with the arguments `btl_tcp_if_include` and an indicator for the local network (`lo`). This is a whole list of jargon, which may be hard to parse if you barely installed the AMUSE package, but maybe it shows that some of the things we do are not always as well supported by local operating systems as we would wish. There are quite a few other complex message-passing and scheduling issues if you desire to

run AMUSE on larger machines, but since the architectures of such machines change within the mean time scale of a PhD (and on the time scale of a new release of this book), we will not dwell further onto this topic.

Bibliography

- Aarseth, Sverre J. 1999. From NBODY1 to NBODY6: The Growth of an Industry. *PASP*, **111**(765), 1333–1346.
- Aarseth, Sverre J. 2011 (Feb.). *NBODY Codes: Numerical Simulations of Many-body (N-body) Gravitational Interactions*. Astrophysics Source Code Library, record ascl:1102.006.
- Allen, Gabrielle, Bengert, Werner, Dramlitsch, Thomas, Goodale, Tom, Hege, Hans-Christian, Lanfermann, Gerd, Merzky, André, Radke, Thomas, & Seidel, Edward. 2001. Cactus grid computing: Review of current development. *Pages 817–824 of: Euro-Par 2001 Parallel Processing: 7th International Euro-Par Conference Manchester, UK, August 28–31, 2001 Proceedings* 7. Springer.
- Backus, John. 1978. *The History of Fortran I, II, and III*. New York, NY, USA: Association for Computing Machinery. Page 2574.
- Beck, K., & Andres, C. 2004. *Extreme Programming Explained: Embrace Change (2Nd Edition)*. Addison-Wesley Professional.
- Binney, James, & Tremaine, Scott. 1987. *Galactic dynamics*.
- Borgdorff, Joris, Falcone, Jean-Luc, Lorenz, Eric, Bona-Casas, Carles, Chopard, Bastien, & Hoekstra, Alfons G. 2013. Foundations of distributed multiscale computing: Formalization, specification, and analysis. *Journal of Parallel and Distributed Computing*, **73**(4), 465–483.
- Calaprice, A. 2002. *Dear Professor Einstein: Albert Einstein's Letters to and from Children*. Prometheus.
- Carol, L. 1871. *Through the Looking-Glass, and What Alice Found There*. Macmillan.
- Chopard, Bastien, Borgdorff, Joris, & Hoekstra, Alfons. 2014. A framework for multi-scale modelling. *Philosophical transactions. Series A, Mathematical, physical, and engineering sciences*, **372**(08).
- Dorman, .F. 2006. The career of Senenmut from Hatsehsut. *Pages 107–109 of: From Queen to Pharaoh Ed. C. Roehrig*.
- Dorman, P. F. 1991. *The Architecture and Decoration of Tombs 71 and 353*. Metropolitan Museum of Art Egyptian Expedition, New York.
- Fryxell, B., Olson, K., Ricker, P., Timmes, F. X., Zingale, M., Lamb, D. Q., MacNeice, P., Rosner, R., Truran, J. W., & Tufo, H. 2000. FLASH: An Adaptive Mesh Hydrodynamics Code for Modeling Astrophysical Thermonuclear Flashes. *ApJS*, **131**(1), 273–334.
- Fryxell, B., Olson, K., Ricker, P., Timmes, F. X., Zingale, M., Lamb, D. Q., MacNeice, P., Rosner, R., Truran, J. W., & Tufo, H. 2010 (Oct.). *FLASH: Adaptive Mesh Hydrodynamics Code for Modeling Astrophysical Thermonuclear Flashes*. Astrophysics Source Code Library, record ascl:1010.082.
- Goodale, Tom, Allen, Gabrielle, Lanfermann, Gerd, Massó, Joan, Radke, Thomas, Sei-

- del, Edward, & Shalf, John. 2003. The Cactus Framework and Toolkit: Design and Applications. *Pages 197–227 of: Palma, José M. L. M., Sousa, A. Augusto, Don-garra, Jack, & Hernández, Vicente (eds), High Performance Computing for Computational Science — VECPAR 2002*. Berlin, Heidelberg: Springer Berlin Heidelberg.
- Gosling, J., Joy, B., & Steele, G.L. 1996. *The Java Language Specification*. 1st edn. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc.
- Gosling, J., Joy, B., Steele, G. L., Bracha, G., & Buckley, A. 2014. *The Java Language Specification, Java SE 8 Edition*. 1st edn. Addison-Wesley Professional.
- Gropp, William, Lusk, Ewing, Doss, Nathan, & Skjellum, Anthony. 1996. A high-performance, portable implementation of the MPI message passing interface standard. *Parallel Computing*, **22**(6), 789–828.
- Holmberg, Erik. 1941. On the Clustering Tendencies among the Nebulae. II. a Study of Encounters Between Laboratory Models of Stellar Systems by a New Integration Procedure. *ApJ*, **94**(Nov.), 385.
- Hunter, John D. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science and Engineering*, **9**(3), 90–95.
- Iwasawa, M., Fukushige, T., Nitadori, D., & Makino, J. 2016. Framework for Developing Particle Simulators (FDPS): Parallel Particle-Mesh Simulations Made Easy. *Publications of the Astronomical Society of Japan*, **68**(3), 54.
- Maassen, J., & Bal, H. E. 2007. Smartsockets: solving the connectivity problems in grid computing. *Pages 1–10 of: HPDC '07: Proceedings of the 16th international symposium on High performance distributed computing*. ACM. 1272368.
- Makino, Junichiro, & Aarseth, Sverre J. 1992. On a Hermite Integrator with Ahmad-Cohen Scheme for Gravitational Many-Body Problems. *Publ. Astr. Soc. Japan*, **44**(Apr.), 141–151.
- Ossendrijver, M. 2016. Ancient Babylonian astronomers calculated Jupiter’s position from the area under a time-velocity graph. *Science*, **351**(Jan.), 482–484.
- Pelupessy, F. I., van Elteren, A., de Vries, N., McMillan, S. L. W., Drost, N., & Portegies Zwart, S. F. 2013. The Astrophysical Multipurpose Software Environment. *A&A*, **557**(Sept.), A84.
- Portegies Zwart, S., McMillan, S. L. W., van Elteren, E., Pelupessy, I., & de Vries, N. 2013. Multi-physics simulations using a hierarchical interchangeable software interface. *Computer Physics Communications*, **183**(Mar.), 456–468.
- Portegies Zwart, Simon. 2011 (July). *AMUSE: Astrophysical Multipurpose Software Environment*. Astrophysics Source Code Library, record ascl:1107.007.
- Portegies Zwart, Simon. 2020. The ecological impact of high-performance computing in astrophysics. *Nature Astronomy*, **4**(Sept.), 819–822.
- Portegies Zwart, Simon, McMillan, Steve, Harfst, Stefan, Groen, Derek, Fujii, Michiko, Nualláin, Breannán Ó., Glebbeek, Evert, Heggie, Douglas, Lombardi, James, Hut, Piet, Angelou, Vangelis, Banerjee, Sambaran, Belkus, Houria, Fragos, Tassos, Fregeau, John, Gaburov, Evghenii, Izzard, Rob, Jurić, Mario, Justham, Stephen, Sottoriva, Andrea, Teuben, Peter, van Bever, Joris, Yaron, Ofer, & Zemp, Marcel. 2009. A multiphysics and multiscale software environment for modeling astrophysical systems. *New Astron.*, **14**(4), 369–378.
- Ritchie, D. M. 1993a. The Development of the C Language. *SIGPLAN Not.*, **28**(3),

- 201–208.
- Ritchie, D.M. 1993b. The Development of the C Language. *Pages 201–208 of: The Second ACM SIGPLAN Conference on History of Programming Languages*. HOPL-II. New York, NY, USA: ACM.
- Rubincam, David Parry. 2000. Radiative Spin-up and Spin-down of Small Asteroids. *Icarus* , **148**(1), 2–11.
- Springel, Volker. 2000 (Mar.). *GADGET-2: A Code for Cosmological Simulations of Structure Formation*. Astrophysics Source Code Library, record ascl:0003.001.
- Springel, Volker, Yoshida, Naoki, & White, Simon D. M. 2001. GADGET: a code for collisionless and gasdynamical cosmological simulations. *New Astron.* , **6**(2), 79–117.
- Stroustrup, B. 2013. *The C++ Programming Language*. 4th edn. Addison-Wesley Professional.
- van Rossum, Guido. 1995a (Apr.). *Extending and embedding the Python interpreter*. Report CS-R9527. pub-CWI, pub-CWI:adr.
- van Rossum, Guido. 1995b (Apr.). *Python library reference*. Report CS-R9524. pub-CWI, pub-CWI:adr.
- van Rossum, Guido. 1995c (Apr.). *Python reference manual*. Report CS-R9525.
- van Rossum, Guido. 1995d (Apr.). *Python tutorial*. Report CS-R9526.
- Van Rossum, Guido, & Drake, Fred L. 2009. *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace.
- Veen, Lourens E., & Hoekstra, Alfons G. 2020. Easing Multiscale Model Design and Coupling with MUSCLE 3. *Pages 425–438 of: Krzhizhanovskaya, Valeria V., Závodszky, Gábor, Lees, Michael H., Dongarra, Jack J., Sloot, Peter M. A., Brissos, Sérgio, & Teixeira, João (eds), Computational Science – ICCS 2020*. Cham: Springer International Publishing.
- Wang, Long, Iwasawa, Masaki, Nitadori, Keigo, & Makino, Junichiro. 2020. PETAR: a high-performance N-body code for modelling massive collisional stellar systems. *MNRAS* , **497**(1), 536–555.
- Wilkinson, C.K. 1979. Egyptian Wall Paintings. *Pages 24–25 of: The Metropolitan Museum’s Collection of Facsimiles*. The Metropolitan Museum of Art Bulletin, vol. 36.
- Zuse, K. 1967. Rechnender Raum. *Elektronische Datenverarbeitung*, **8**, 336–344.
- Zuse, K. 1993. *The Computer – My Life*.

Chapter 2

Fundamental concepts

When I let go of what I am,
I become what I might be.

Modern Taoist interpretation of
a potential saying by Laozi, but
no ancient Chinese version exists
in the Tao Te Ching ([Laozi, 2009](#)).

We live in an era of multi-messenger astronomy, meaning that various instruments operational in a wide range of the electromagnetic spectrum cooperate to collect data. In computational astrophysics we have the concepts of multi-scale and multi-physics modelling, which are the equivalents of multi-messenger astronomy. Here, we discuss the fundamental aspects that enables multi-messenger computational astrophysics. Fundamental parts include data handling, communication, unit conversion, and scaling.

2.1 Programming

The majority of code development you will do as a apprentice AMUSE user will be in the Python programming language ([van Rossum, 1995a,b,c,d](#)). In the previous version of this book, we adopted Python version 2.7, this version of the book is written with Python 3.7+ in mind ([Van Rossum & Drake, 2009](#)).

2.1.1 Why Python

Different programming languages, like Fortran ([American National Standards Institute, 1978](#)), C ([Kernighan & Ritchie, 1988](#)), and Python have been the main language of use for us. The first two authors moved to C++ ([Stroustrup, 2013](#)) to write an extensive N-body simulation environment called Starlab ([Hut *et al.*, 2010](#)).

Many of the modules in AMUSE are written in these languages. Although our preferences differ, and we may nowadays prefer C++, Julia ([Bezanson *et al.*, 2017](#)), or Rust ([Steve Klabnik, Carol Nichols, 2023](#)), the main AMUSE package is written in Python.

Python has become one of the most popular and versatile programming languages, it is ease of use, versatility, reasonably efficient, data focused, and it has a great and active development community. In addition, it makes working with unstructured text almost fun, whereas this has always been one of the older languages' weaker points.

Our choice of Python gives AMUSE users access to a powerful, flexible, and widely used language that opens doors to numerous career opportunities and enables them to address a wide range of programming challenges.

For clarity, we briefly review some of the most important style recommendations when you write or develop any scripts on your own. We present these small guidelines to ensure uniform formatting across the code you write. Homogeneous formatting reduces decision making so that you can focus on solving problems. It facilitates teamwork, and improves improves maintainability through adherence to best practices.

2.1.2 Style guide

Here, we cover the most important aspects of the Python Enhancement Proposal (PEP 8, [van Rossum *et al.*, 2001](#)). Some further style recommendations are available in [Warsaw \(2022\)](#), which we recommend reading. The Python style guide includes recommendations for indentation, imports, the use of whitespace, naming conventions, comments, docstrings, and basic syntax preferences. Keep in mind though that these are just guidelines. When your specific requirement needs you to abandon them, abandon them.

Resist the urge to put in more than one import per line. But group them in order of the standard Python libraries, third party libraries, and as last your local sources.

```
import os
import numpy as np
from amuse.units import units
```

The first two lines in this snippet should look familiar. The third line directly uses a specific module from the AMUSE framework. This is a module that allows us to convert units from one set to another. The unit module is quite handy and general, and AMUSE was the first astronomical code that incorporated a general unit module. We will go in some depth in explaining the unit module in [Section 2.3](#).

Parameters in your script can be provided with units using the 'pipe' operator, for example a parameter `mass` can be provided with the mass of 1 solar mass by

```
from amuse.units import units
mass = 1 | units.MSun
```

Avoid the use of additional whitespace between braces, brackets and parenthesis. Please write

```
sun_and_jupiter = Particles(name=["Sun", "Jupiter"], m=(1, 0.001) | units.MSun)
```

rather than this **bad example**:

```
SunAndJupiter = Particles ( name = [ "Sun", "Jupiter" ], m = ( 1, 0.001 ) | units.
    MSun)
```

For functions, variables and methods, we use *snake_case*: lower-case separated by an underscore, and keep lines under 80 characters. Note that, in accordance with PEP 8, we prefer white spaces after commas (',') and double columns (':'), while all operators are surrounded by white space, such as in

```
mass_total = (1 | units.MSun) + (1 | units.MJupiter)
```

except for equal signs ('=') in function calls.

For classes we use *PascalCase*, where all words begin with a capital letter

```
class MilkyWayGalaxy():
```

whereas constants are all in capitals with underscores separating the words

```
# This is the distance from the Sun to the Galactic center.
SUN_GALACTIC_DISTANCE = 8.5 | units.kpc
```

Comments in a code are complete sentences. But at the start of a function we use docstrings

```
import numpy as np
from amuse.units import constants

def semi_to_orbital_period(semi_major_axis, mass_total):
    """
    Convert semi-major axis to orbital period
    """
    return 2 * np.pi * (semi_major_axis**3 / (constants.G * mass_total)).sqrt()
```

We use 4 spaces per indentation level. Note that portable document format (pdf) does not support whitespace, and those are not properly copy-pasted from the book to your Python environment.

For comparisons we use 'is' for singletons, and '==' for values

```
if x is None:
    pass
```

We regularly include Python snippets in the text. Sometimes they are written as standalone sentences or comments, and sometimes they are taken directly from one of the larger scripts in the example package. In the latter case, the code lines are associated with line numbers. These line numbers refer directly to the appropriate example script (although these could change if and when the examples are revised).

2.2 Internal data handling and communication

Data is a collection of facts in the form of bits of information, often expressed in numbers, text, images, or sound. It represents information that can be quantified and used to understand patterns, trends, and relationships in the world around us. It serves as the foundation for decision-making and leads to deeper insight. Data can be processed, manipulated, and analyzed to extract insights. As the amount and complexity of data, in science and beyond, continues to grow (probably exponentially), the need for improved data handling increases. We have no ready answer to the complexities of data analysis; we consider its handling a science, not an art.

2.2.1 Particles and Particle sets

In our experience, most of the syntactic and mathematical features of Python (conditionals, loops, arithmetic operations, etc.—even the rigid use of indentation to define program structure)—are broadly intelligible to most programmers, regardless of background. However, some more complex data structures may be unfamiliar to readers used to less structured programming languages, such as C or Fortran. Accordingly, we pause here to elaborate on this topic in AMUSE, using some important data structures as templates.

A Python class is a data structure together with a collection of associated functions, or methods, designed to manipulate the data. A particularly simple AMUSE class is a `Particle`, which contains only basic particle attributes, such as mass, position, velocity, etc., as well as a randomly generated key that uniquely identifies the particle in the AMUSE system. We can access its contents directly, and expand it to store all sorts of useful information.

```
from amuse.datamodel import Particle
from amuse.units import units

p = Particle()
p.mass = 1.0 | units.MSun
p.mass2 = 42 | units.kg
p.name = "Christine Chapel"
p.rank = "Nurse"
print(p)
```

The `particle` class is not very useful in and of itself, but it forms the basis for a vitally important class called a **particle set**, which is basically an array of particle objects, with conventions regarding content and a collection of methods providing information on and control over the data. Particle sets are used to transfer data from one code to another, and for eventual data processing and analysis. An element of a **particle set** is expected to have certain attributes, such as **mass**, **position**, **velocity**, **radius**, and possibly others. The built-in methods assume that these data exist.

You can generate a (rather useless, for now) particle set by using the following lines

```
from amuse.datamodel import Particles

particles = Particles(number_of_particles)
print(particles)
```

This prints the most fundamental ingredient in a particle set; the **key**, which identifies each particle. Note here the difference between the singular and the plural forms for declaring a particle and a particle set.

```
          key
          -
=====
5512314219721684995
8353633730176500107
9653245888285946456
9457276936209163492
14505666692805646583
14003077150553218257
=====
```

There is not much one can do with such a simple bunch of particles, except perhaps to use it as a very expensive random number generator. However, we can easily extend it in useful ways. We can create a particle set of 10 particles as follows:

```
from amuse.datamodel import Particles

particles = Particles(10)
```

We can then set up attributes in various ways:

```
particles.mass = 1 | units.MSun
```

sets all masses, while

```
particles[3].mass = 42 | units.kg
```

sets just the mass for the fourth particle (Python starts counting at 0). We could also have set the masses (or positions, etc.) in the **Particles** creation call, with

```
particles = Particles(10, mass=1 | units.MSun)
```

Some more useful ways of setting particle set attributes are presented below.

In some cases, one would like to expand the attribute list of a particle with an array. This can happen when one would like to add information about a chemical network to a particle, or some other attributes that normally is associated with particle data. Adding abundances, for example, for Hydrogen, Helium, Lithium and Iron could be realized as follows

```
particles.add_vector_attribute("abundance", ("H", "He", "Li", "Fe"))
particles.abundance = [0.75, 0.24, 0.005, 0.005]
```

In the first line we add a vector attribute called “abundance” to the particle set, and

specify the array of names for this attribute. In the second line we initialize the values for all the 10 particles with the same chemical composition.

Once the basic attributes are set, the methods associated with the `particle set` provide commonly used functionality. For example,

```
energy_kin = particles.kinetic_energy()
```

computes the total kinetic energy of the particle set, while

```
energy_pot = particles.potential_energy()
```

computes the total potential energy. Similarly,

```
com = particles.center_of_mass()
```

calculates the center of mass of the particle set, while

```
particles.scale_to_standard()
```

places the set in the center of mass frame and scales it to a standard configuration (see Section 4.1.2.1). Many AMUSE functions take a particle set as input and return analytical information on it.

You can ask Python to list the methods associated with a class by using the `dir` function:

```
dir(Particles)
```

will provide a list of all the methods available in the `Particles` class. (Note the Python convention that names beginning and ending with double underscores are intended for private use, even though you can see them and in principle use them yourself—not recommended!) More extended information can be obtained with

```
help(Particles)
```

which provides a detailed description of the methods in the class.

Even here, though, Python asserts its inherent flexibility. Unlike classes in more strongly typed languages, Python classes are customizable. You can add your own variables at will for your own purposes—just don't expect them to be properly managed or preserved by the built-in methods! Such additional attributes are also ignored by the specific modules, and will not be copied back and forth between module and local script. Just keep in mind that they have to be handled explicitly by the script writer.

2.2.2 Grid-based data

Technically grids are conceptually quite different than particles, but in principle they can represent the same information, at least numerically. Grids are also used in gravity codes (such as in tree codes (Barnes & Hut, 1986) and particle-mesh solvers (Hockney & Eastwood, 1988)), and they are quite common for hydrodynamics solvers. In principle the same information can be conveyed with particles, or with grids, but for a wide variety of reasons grids often turn out more complex because they are not necessarily

Cartesian.

Voronoi grids, for example, represent a tessellated space, whereas a KDTree-type grid or a refined grid have different data structures. These differences in the fundamental data structure makes it difficult to unify the grid-based data structures. We therefore project grids onto the already existing particle data structure.

Examples of grid-based codes include Athena and FLASH. Both use adaptive grids, although, this is not how we initialize them. These grids are most easily initialized using a Cartesian grid, after which the code internally readjusts the grid structure to reflect the properties of the gas (often the density or temperature structure).

Grid-based hydrodynamical solvers or radiative transfer codes often handle the data locally in the particular code, rather than in the AMUSE script. An example is the `Torch` script [Wall *et al.* \(2020a,b\)](#), which combines the grid-based magnetohydrodynamics solver `FLASH` with AMUSE. The grid-based data in `Flash` is handled, saved, and stored in the code. The AMUSE script can in principle touch the data, but this leads to complex data consistency responsibility issues, performance loss, and unnecessary data replication.

A simple grid for the hydro-code Athena can be initialized as follows

```
from amuse.units import generic_unit_system
from amuse.community.athena import Athena

instance = Athena()
instance.parameters.x_boundary_conditions = ("periodic", "periodic")
instance.parameters.mesh_length = (8, 1, 1) | generic_unit_system.length
instance.parameters.mesh_size = (8, 1, 1)
grid = instance.get_extended_grid()
instance.initialize_grid()
print(grid.rho)
```

A more generic way of making a grid is

```
def make_grid(number_of_grid_cells,
              length,
              constant_hydrogen_density,
              inner_radius,
              outer_radius
              ):
    grid = Grid.create([number_of_grid_cells] * 3,
                      length.as_vector_with_length(3)
                      )
    grid.radius = grid.position.lengths()
    grid.hydrogen_density = constant_hydrogen_density
    grid.hydrogen_density[grid.radius <= inner_radius] = 0 | units.cm ** -3
    grid.hydrogen_density[grid.radius >= outer_radius] = 0 | units.cm ** -3
    return grid
```

We can plot the particles of a gaseous distribution simply using matplotlib, as demonstrated in Figure 2.1.

In Figure 2.2 we show three representations of a gaseous distribution (right), as a Voronoi tessellated structure (left) and as an adaptive Cartesian grid (middle).

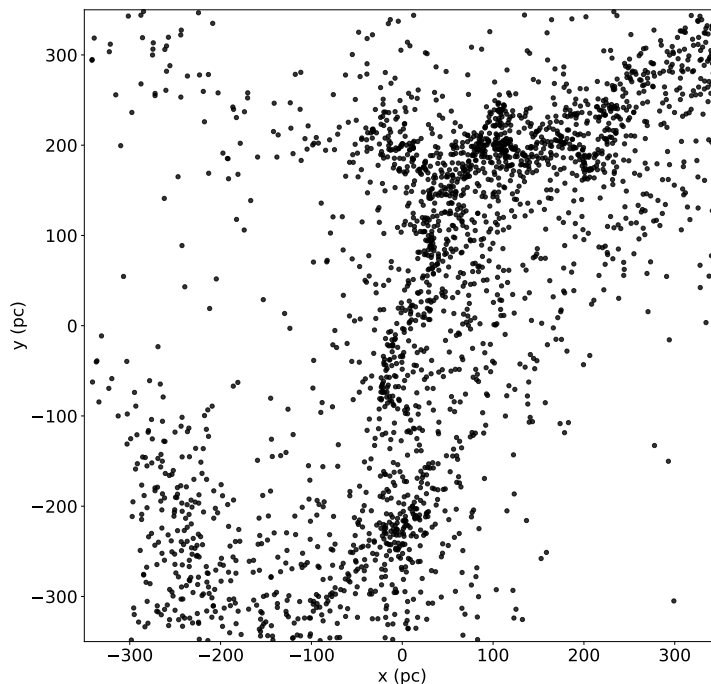


Figure 2.1: The calculation was performed using the Torch package in AMUSE. The projection is 700×700 pc. Tessellated, Adaptive grid and the gaseous distributions are presented in Figure 2.2. The script to generate this figure can be found in `ae.hydro.voramr_logic_progression_plots`.

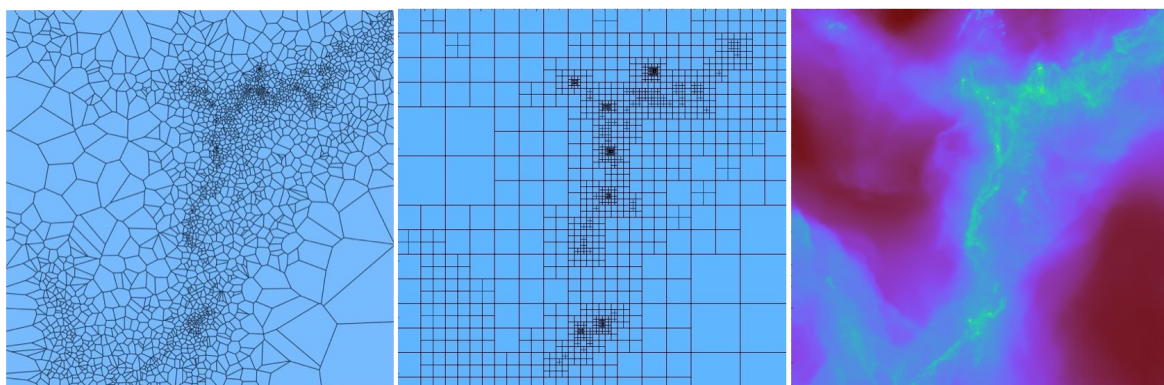


Figure 2.2: Voronoi tessellation (left) (Lewis *et al.*, 2023), adaptive Cartesian grid (middle) and a gaseous representation (right) from a simulation performed using FLASH. The projection is 700×700 pc. in Figure 2.1 we show the particle gas distribution of the same filament. The script to generate this figure can be found in `ae.hydro.voramr_logic_progression_plots`.

We construct and plot the Voronoi tessellation from the particle gas `particle_data`, using the `scipy` package `Voronoi`.

```
from scipy.spatial import Voronoi, voronoi_plot_2d
voronoi_data = Voronoi(
    particle_data, furthest_site=False, qhull_options="Qbb Qc Qz Gc",
)
voronoi_plot_2d(voronoi_data, ax, show_vertices=False)
```

The refined grid is plotted conveniently using the YT package (see <https://yt-project.org/doc/index.html>).

```
AMR_data = yt.load("../data/lv112-VorAMR_hdf5_chk_0001")
plt = yt.ProjectionPlot(AMR_data, "z", "bgpt")
plt.set_cmap(field=("flash", "bgpt"), cmap="arbre")
plt.annotate_grids(cmap="black_only")
```

Several codes make use of grids. Some of them solve for gravity, such as the various tree codes, and others solve for either hydrodynamics or radiative transfer. We will have more to say on these codes in their respective chapters: Chapters 4, 6 and 7.

2.2.3 Data handling between code and script

Let's imagine that particle information (mass, position, velocity, etc.) is stored in a particle set called `bodies`, which is part of local memory and lives in the user Python script. This data does not naturally live in the selected worker code, but the essential parts can be copied to the code as needed. For a stellar evolution code this might be the mass and radius, and for an N-body code this could also include position and velocity. This local information is not automatically updated after the code has been run using `{code}.evolve_model()`. Here `{code}` may be the instantiation of any type of code for evolving stars or integrating equations of motion.

To access the new data, we could copy it directly from the remote code by referencing `{code}.particles`, which lives in the code interface and is updated by `{code}.evolve_model()`, but each access will cause AMUSE to open a one-time MPI or sockets connection to the running code where the remote data live, which can be slow in terms of compute time because it requires the construction of the connection. More importantly, it is less explicit. Note that some codes may have various types of particles. This is not uncommon in particle-based hydrodynamics solvers, which may make a distinction between `dm_particles` for dark matter, and `gas_particles` for the gas.

A more efficient way to communicate with a running code is realized by creating a *channel* to the remote data of the running worker code. A communication channel for (in this case) a gravity code is declared with

```
channel_from_gravity_to_framework = gravity.particles.new_channel_to(bodies)
```

Here `gravity` is a pointer to the running code. To copy the particle data from the running code to the local Python framework, we say

```
channel_from_gravity_to_framework.copy()
```

Now the masses, positions, and velocities of the remote data in the worker module are duplicated in the local `bodies` particle set in the framework (Python) script. The local data (in `bodies`) and the remote data (in `gravity.particles`) are consistent. The `copy()` command copies all data, but to tune performance or to prevent overwriting some local data, one can choose to copy only part of the data. For example, to update only the positions, but not the masses, velocities, or other attributes, we could write:

```
channel_from_gravity_to_framework.copy_attribute(["x", "y", "z"])
```

We could have arranged for the local data to be automatically synchronized with the worker data on each return from the worker code, but for the sake of user control and efficiency, we have opted instead for manual synchronization.

2.2.4 Storing and Recovering Data

We can store particle data in a file for further and future analysis. It is advisable to synchronize the local `bodies` with the code data (see Section 2.2.3) before writing them to a file. A snapshot can be saved to a file named `filename` as follows:

```
write_set_to_file(bodies, filename)
```

Optionally, a “`format={format}`” argument indicates the format in which the snapshot is stored—by default “`amuse`”, which uses `hdf5`. This format is commonly used within the astronomical community; it provides a self-descriptive binary format with the potential of storing hierarchical data structures ([The HDF Group, 2000-2010](#)). Other recognized types include “`starlab`” (the internal StarLab `dyn` and `star` formats [Portegies Zwart *et al.* \(1999\)](#); [Hut *et al.* \(2010\)](#)), “`csv`” (comma-separated-values), and plain text (“`txt`”).

Time is not an attribute of the particle set, but rather of the code, in this case the N -body integrator. As a consequence, the age of a particle is not stored by default. This generally saves data and the historical context stems from the idea that all stars are born at the same time. Some codes do not even allow for particles to have different ages, but in AMUSE we are not bound by these limitations. To save the time of a snapshot, we can use:

```
write_set_to_file(bodies.savepoint(0 | model_time.unit), filename)
```

Here, the phrase `bodies.savepoint` ensures that the time is stored with the snapshot. A stored output file can be read back to memory:

```
bodies = read_set_from_file(filename)
```

If time was saved with the snapshot, it can be recovered:

```
model_time = bodies.get_timestamp()
```

For storing the ages of individual particles, one can simply add a new attribute to the particle set:

```
bodies.age = 0 | units.Myr
```

or to an individual particle:

```
Sun.age = 4.5678 | units.Gyr
```

Multiple snapshots can be stored in a single file. In that case, one can iterate over the individual snapshots using

```
many_snapshots = read_set_from_file(filename)
for snapshot in list(many_snapshots.history):
    print(f"The number of bodies in this snapshot is: {len(snapshot)}")
```

For runs with a large number of particles, or containing many snapshots, these files can become quite large. One solution is to split the output into smaller pieces and process them separately. Sometimes, however, this is not desirable. In such cases, one can keep the data on disk by keeping the file open while reading:

```
bodies = read_set_from_file(
    filename,
    copy_history=False,
    close_file=False,
)
```

Alternatively, one could write compound particle sets to a single file and recover them with:

```
gas, stars = read_set_from_file(
    filename,
    names=["gas", "stars"],
    copy_history=False,
    close_file=False,
)
```

Yet another way, which is often used in planetary systems, is to make a distinction in the particle set by introducing an extra attribute:

```
stars.name = "star"
planets.name = "planet"
moons.name = "moon"
bodies = stars + planets + moons
write_set_to_file(filename, bodies)
```

After reading the file one can split the particle set:

```
bodies = read_set_from_file(filename)
stars = bodies[bodies.name == "star"]
```

and similarly for the other particle subsets.

2.3 Converting between code units and SI

2.3.1 Scaling standard units to internal code units

Units are fully supported in AMUSE. Running one of the interfaced high-performance codes with units is not difficult. For intrinsic dimensionless solvers including units results in the loss of the scale-free characteristics of the problem. This is only a problem in gravitational dynamics, which can be addressed elegantly as a scale-free problem using Hénon units (Hénon, 1971). For stellar evolution, unit conversion is generally not an issue because both the Henyey and the parameterized stellar evolution codes already work in units that naturally convert to SI. Similar arguments hold for hydrodynamics solvers and radiative transfer codes, but they tend to be written for specific sets of units. We desire to achieve coherency and consistency in the use of units, and therefore introduce the converter class. If you are unfamiliar with Python classes, don't despair, as we provide a quick overview in Appendix C.

The units in an N -body system are specified by providing any two of the mass, length, and time scales of the problem. As a practical matter, conversion to other systems requires the creation of a `converter`, which is a Python class specifically designed to keep track of units. We can construct a converter with mass unit $1 M_{\odot}$ and spatial unit 1 au as follows:

```
converter = nbbody_system.nbbody_to_si(1 | units.MSun, 1 | units.au)
```

However, the following line

```
converter = nbbody_system.nbbody_to_si(72 | units.kg, 175 | units.cm)
```

which happens to be the weight and height would work equally well. In this last example we really did use the International System of Units, whereas for most astronomical simulations, we are more likely to use solar masses, millions of years, and parsecs. Nevertheless, the conversion system is still generically called `nbbody_to_si`.

A system in physical units can be converted back to N -body units using the same converter. For example, converting the positions back to N -body units can be accomplished with

```
positions_in_nbbody_units = converter.to_nbbody(bodies.position)
```

or back to SI units:

```
positions_in_si_units = converter.to_si(positions_in_nbbody_units)
```

More information about alternative converters can be found in Section 2.3.2

2.3.2 Alternative converters

Instead of using a mass and a time, or a mass and a length-scale, to specify the set of units it is also possible to use instead luminosity and density, or any other dimensionally independent quantities. The routine for specifying a different set of units can be imported by

```
from amuse.units.generic_unit_converter import ConvertBetweenGenericAndSiUnits
```

We can call this routine with (for example) luminosity `L` and density `rho`, would look as follows:

```
converter = ConvertBetweenGenericAndSiUnits(L, rho, constants.G)
```

Alternatively, it can be called with any set of independent units:

```
converter = ConvertBetweenGenericAndSiUnits(
    1.0 | units.kpc,
    1.0e10 | units.MSun,
    constants.G,
)
```

or if `Rvir` is virial radius and `rho` is density,

```
converter = ConvertBetweenGenericAndSiUnits(Rvir, rho, constants.G)
```

2.3.3 Changing Parameters of the simulation

Often the functionality of a community code needs to be modified, for example to improve accuracy or set some other parameter. This is controlled by internal parameters within the code. Changing those internal parameters is always possible, but we want to avoid modifying and recompiling the code. For this reason, many internal code parameters can be queried and (re)set. The names of these parameters may be the same for some codes, but completely different for others. Fortunately, as described in Section 2.2.1, Python and the AMUSE interface have the ability to provide help on these matters. The command

```
dir(Ph4)
```

provides a extensive list of all the ingredients of the direct N -body code `Ph4` (see Chapter 4 for a more detailed description of N -body codes). This command can be given for each code, such as

```
dir(Mocassin)
```

The same command with the code `Bhtree` will provide a different list of functionalities for the tree code, and equally wise for the radiative transfer code `Mocassin`. More extended information can be obtained with

```
help(Ph4)
```

A list of parameters for `Ph4`, and their default values, can be obtained for the instantiation of a particular code, such as for the same gravity solver

```
gravity = Ph4()
```

we can get information about the parameter settings through

```
gravity.parameters?
```

A list of all available parameters (and their default settings) of this particular code is provided by

```
print(gravity.parameters)
```

but the same constructs such as `help` and `dir` can also be applied to the instantiation of the code, or to its member functions.

The parameters used in two seemingly similar codes may have different names. This sounds very confusing, but it reflects the diversity of the authors of the various interfaces. Our philosophy here is to try to reserve identical names for parameters that do exactly the same thing, but because many codes have slightly different functions or objectives, it is hard to be consistent and clairvoyant at the same time. In Section 4.3.6

Some parameters control the actual working of a code. A smoothed-particles hydrodynamics code, such as `Fi()` allows the user to control the number of nearest neighboring particles to determine the local thermodynamical parameters. This can be tuned by setting the parameter from the default value of 32 to, one can write:

```
hydro.parameters.targetnn = 64
```

The number of free parameters varies a lot depending on the code. The stellar evolution code `Seba()` has 11 parameters, the `Ph4()` *N*-body code has 20, and the smoothed-particles hydrodynamics code `Fi()` has 60. Having more (or fewer) parameter does not necessarily say much about the reliability of the code, but it does say something about the effort we took in making these parameters accessible by the user from the Python interface.

The best way to learn about the various parameter settings, and the possibilities of a specific code is by looking at good old-fashioned source code, as all parameter settings are defined in the interface file of each community module in the AMUSE source code. For the `Hermite` package, for example, the parameters can be found in the method `define_parameters()` in the `amuse.community.hermite.interface.Hermite` class.

2.4 Tricks with units

2.4.1 Setting printing strategies

Printing quantities with units can be awkward, because the units often come out in undesired form: $1M_{\odot}$ might be printed in grams, or 1 pc in centimeters. To prevent this unwanted behavior, you could explicitly convert to your desired units, e.g.

```
x = 10 | units.cm
print(f"x={x.in_(units.parsec)}")
```

but this is quite cumbersome and inconvenient for routine use. An alternative is to set a preferred printing strategy. For example, if your script performs all calculations in

solar masses, solar radii, and megayears, you can indicate your preferred set of units with

```
set_printing_strategy(
    "custom",
    preferred_units=[units.MSun, units.RSun, units.Myr],
    precision=6,
    prefix="",
    separator=" ",
    suffix="]",
)
```

This will ensure that all numbers will be printed with 6 decimal places and with the units in brackets. But when you perform a galaxy collision simulation you may prefer

```
set_printing_strategy(
    "custom",
    preferred_units=[units.MSun, units.kpc, units.Gyr]
)
```

To print in other unit systems, such as N-body units, a converter must be provided in the argument list, for example, `nbody_converter = converter` (see Section 2.3 for details).

2.4.2 The command-line argument parser

Production scripts usually require tuning for application to a specific problem, and it is desirable to be able to change a script's input parameters, such as the number of stars, their masses or metallicity, the virial radius of a cluster, or the position of a black hole, without having to edit it every time. These parameters are traditionally set on the command line and subsequently parsed by the production script.

One solution in Python for managing command-line arguments is the argument parser

```
import argparse
```

Code example 2.1 presents a usage example of this command-line argument parser, where we allow the stellar mass `M`, metallicity `z`, and the end time of the simulation `model_time` to have the desired units.

Bibliography

- American National Standards Institute. 1978. *American National Standard Programming Language FORTRAN*. Technical Report ANSI X3.9-1978. American National Standards Institute, New York, NY.
- Barnes, Josh, & Hut, Piet. 1986. A hierarchical $O(N \log N)$ force-calculation algorithm. *Nat*, **324**(6096), 446–449.
- Bezanson, Jeff, Edelman, Alan, Karpinski, Stefan, & Shah, Viral B. 2017. Julia: A fresh approach to numerical computing. *SIAM review*, **59**(1), 65–98.

```

50 def new_argument_parser():
51     parser = argparse.ArgumentParser(
52         formatter_class=argparse.ArgumentDefaultsHelpFormatter,
53     )
54     parser.add_argument(
55         "-m",
56         type=units.MSun,
57         dest="mass",
58         default=1.0 | units.MSun,
59         help="stellar mass",
60     )
61     parser.add_argument(
62         "-t",
63         type=units.Myr,
64         dest="time_end",
65         default=4700.0 | units.Myr,
66         help="end time of the simulation",
67     )
68     parser.add_argument(
69         "-z",
70         dest="metallicity",
71         type=float,
72         default=0.02,
73         help="metallicity",
74     )
75     return parser
76
77
78 def main(**kwargs):
79     stellar_minimal(**kwargs)
80
81
82 if __name__ == "__main__":
83     arguments = new_argument_parser().parse_args()
84     main(**arguments.__dict__)

```

Code example 2.1: Command-line parser for a stellar evolution model and the main routine for calling the function listed in Code example 5.3 in Section 5.2.2. This snippet is taken from `amuse.examples.stellar_minimal`.

- Hénon, M. H. 1971. The Monte Carlo Method (Papers appear in the Proceedings of IAU Colloquium No. 10 Gravitational N-Body Problem (ed. by Myron Lecar), R. Reidel Publ. Co. , Dordrecht-Holland.). *Ap&SS* , **14**(1), 151–167.
- Hockney, R.W., & Eastwood, J.W. 1988. *Computer Simulation Using Particles*. Adan Hilger Ltd., Bristol, UK, 540 pp.
- Hut, Piet, McMillan, Steve, Makino, Jun, & Portegies Zwart, Simon. 2010 (Oct.). *Starlab: A Software Environment for Collisional Stellar Dynamics*. Astrophysics Source Code Library, record ascl:1010.076.
- Kernighan, Brian W., & Ritchie, Dennis M. 1988. *The C Programming Language*. 2 edn. Englewood Cliffs, NJ: Prentice Hall.
- Laozi. 2009. *Tao Te Ching*. Dao de jing. English. London: Frances Lincoln Limited. Translated from Chinese. Subtitle on cover: "An illustrated journey".
- Lewis, Sean, McMillan, Stephen, Mac Low, Mordecai-Mark, Cournoyer-Cloutier,

- Claude, Polak, Brooke, Tran, Aaron, Wilhelm, Maite, Sills, Alison, Klessen, Ralf, & Wall, Joshua. 2023 (Jan.). Star Cluster Formation: The effects of early forming massive stars and building a bridge between Voronoi mesh and block-structured codes. *Page 109.03D of: American Astronomical Society Meeting Abstracts #241*. American Astronomical Society Meeting Abstracts, vol. 241.
- Portegies Zwart, S. F., Makino, J., McMillan, S. L. W., & Hut, P. 1999. Star cluster ecology. III. Runaway collisions in young compact star clusters. *A&A* , **348**(Aug.), 117–126.
- Steve Klabnik, Carol Nichols. 2023. *The Rust Programming Language*.
- Stroustrup, Bjarne. 2013. *The C++ Programming Language*. 4 edn. Upper Saddle River, NJ: Addison-Wesley.
- The HDF Group. 2000-2010. *Hierarchical data format version 5*.
- van Rossum, Guido. 1995a (Apr.). *Extending and embedding the Python interpreter*. Report CS-R9527. pub-CWI, pub-CWI:adr.
- van Rossum, Guido. 1995b (Apr.). *Python library reference*. Report CS-R9524. pub-CWI, pub-CWI:adr.
- van Rossum, Guido. 1995c (Apr.). *Python reference manual*. Report CS-R9525.
- van Rossum, Guido. 1995d (Apr.). *Python tutorial*. Report CS-R9526.
- Van Rossum, Guido, & Drake, Fred L. 2009. *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace.
- van Rossum, Guido, Warsaw, Barry, & Coghlan, Nick. 2001 (jul). *PEP 8 – Style Guide for Python Code*. Python Enhancement Proposal 8. Python Software Foundation. Last updated: 2013-08-01.
- Wall, Joshua E., Mac Low, Mordecai-Mark, McMillan, Stephen L. W., Klessen, Ralf S., Portegies Zwart, Simon, & Pellegrino, Andrew. 2020a. Modeling of the Effects of Stellar Feedback during Star Cluster Formation Using a Hybrid Gas and N-Body Method. *ApJ* , **904**(2), 192.
- Wall, Joshua E., Tran, Aaron, Mac Low, Mordecai-Mark, McMillan, Stephen L. W., Klessen, Ralf S., Portegies Zwart, Simon, & Pellegrino, Andrew. 2020b (Mar.). *Torch: Coupled gas and N-body dynamics simulator*. Astrophysics Source Code Library, record ascl:2003.014.
- Warsaw, Barry A. 2022. *GNU Mailman Coding Style Guide*. Accessed: [Insert Date].

Chapter 3

Initial conditions

in nīz bogzarad
This too shall pass.

Solomon – Israel Folklore Archive #126

Any simulation requires a starting point. We call these the initial conditions. Some initial conditions are simple, some are extraordinary complex. The simplest ones are often the most interesting, and lead to the clearest understanding of the complex phenomena that follow through calculation. Many final conditions eventually turn into initial conditions for future simulations. In astrophysics, the ultimate initial conditions are probably the beginning of time and the Universe, whatever they were.

3.1 Starting simulations

The generation of initial conditions for complex multiphysics simulations is a critical step in ensuring accurate and efficient computational modeling. The complexity grows with the demands and realism in the simulations. In multiphysics simulations, where multiple physical phenomena interact simultaneously, the choice of initial conditions have an unprecedented impact the simulation’s stability, convergence, and the interpretability of the results.

The challenge lies in creating a initial conditions that are consistent with the governing equations, boundary conditions, and physical constraints of the system, while, when evolved in time, teach you something about the underlying processes that govern the system’s evolution.

3.1.1 Warning on initial conditions

Every simulation in AMUSE requires initial conditions to start a calculation. Typically these entail choosing dynamical quantities such as particle masses, positions, and velocities, stellar properties such as radii and luminosities, or gas properties such as density, temperature, and composition. These quantities are often the result of some prior calculation, or represent a restart from a previously stored state. In principle, any mathematically consistent set of initial choices is a valid way to start a simulation.

In practice, however, we generally want to use some reasonable, physically motivated starting point for further study.

Some initial conditions are realistic, but most are not—and most of the time that is okay. Realistic initial conditions are the goal of (almost) all researchers, but they are hard to find and generally even harder to explain. Most studies start with somewhat unrealistic initial conditions, with the expectation (or at least the hope) that they capture the fundamental physics necessary to study the desired phenomenon, and that the details of the simulation will rapidly drive the system into some physically more meaningful configuration. In follow-up studies, we can then improve on the earlier choice of initial conditions, say by covering a larger part of parameter space or by zooming in on a region of interest. In some cases, for example the stellar density at the moment of core collapse in an idealized equal-mass star cluster (see Section 4.1.3.2), the evolution is quite insensitive to the details of the initial conditions. In others, however, such as star formation in an interstellar molecular cloud (Chapter 6), the outcome depends crucially on the initial state.

A simple example of unrealistic but widely accepted initial conditions are those used in gravitational N -body simulations of star clusters. Such simulations generally start by setting the number of stars N , and distributing them in space with specified radial (spherically symmetric) density and (truncated Maxwellian) velocity profiles. Most modern studies of star clusters incorporate some sort of (parametrized) stellar evolution, so the initial conditions must then also include stellar masses and radii. The entire system is then scaled (usually by adjusting the velocities) to virial equilibrium (see Section 4.1.2), before high-precision N -body integrators are used to study the dynamical evolution of the system. From an astronomical point of view, these initial conditions are not correct, but they can still be very useful in the study of gravitational dynamics.

More realistic initial conditions for this problem might start with an interstellar gas cloud, which collapses and fragments to form stars (see Section 6.7.4). Once massive stars begin to shine, they produce winds and radiation that cause the remaining gas to be blown away, terminating further star formation and locally leaving behind a star cluster. In this case, no ad hoc specifications of the positions and masses of cluster stars are required. But of course, this only shifts the problem back in time, as the initial conditions for the gas simulation now must specify the (clumpy) density distribution and the (turbulent) motion of the cloud. Those “initial conditions” in turn depend on the large-scale flow of gas in the galaxy, and so on. The fact is that there are no “true” initial conditions for this problem, just convenient milestones in a continuous process. Most generally adopted initial conditions have problems, but doing much better can turn garbage into science, and science into garbage.

AMUSE contains an extensive (and growing) collection of codes to generate initial conditions. Very often one code provides the conditions from which another code continues. For example, when two stars collide, a stellar evolution code might provide the initial density and temperature profiles for the two colliding stars, while a gravitational dynamics code provides their relative positions and velocities. In this way two codes are used to generate the conditions for starting a third code that simulates the hydrodynamics of the collision. In the next subsections we discuss some common initial

conditions used in astrophysical studies, but we focus on the simplest ones, those that are generated by a single rather simple routine.

3.1.2 Variations in starting points

One can imagine several approaches in the generation of initial conditions. A new starting point might be generated from a single free parameter, or it may be the result of a large-scale simulation. Both are found in astronomy; people still start by randomly selecting stellar masses from a power-law mass function (which has 3 degrees of freedom), while others take the end result of a large-scale cosmological simulation to extract a specific aspect which subsequently is modeled in higher resolution or by including different physical processes.

The few-parameter generated approach results in a clean experiment, with the advantage of the easier interpretation, whereas the latter results in more complex relation between initial conditions and eventual simulation results. Which is preferable depends on your question.

We can separate the generation of initial conditions naively in the following categories (in order of increasing complexity):

- In steady-state initialization, we solve a stationary version of the problem to obtain a consistent initial state. Although it's not always clear exactly what consistent means, in our case, it would reflect the masses, positions, and velocities of objects in space, such as a solar system, or as we demonstrate in Section 4.3.1 by generating a cluster of stars, or by realizing a molecular cloud that collapses under its own gravity (Section 6.5).
- By gradually introducing loads and boundary conditions over time we assist a system to transition from simple initial values to more complex states. This ramping techniques is often used when converting the end conditions from one physical domain to start a simulation in another. An example is the conversion of a thermal-equilibrium stellar evolution model to a dynamical equilibrium hydrodynamical model of the same star, as we demonstrate in Section 6.4.2 by simulating the collision between two stars.
- Experimental data or results from simplified models can provide interesting initial conditions. This data-driven approach has become more popular with the availability of large quantities of multi-messenger data. We present a brief data-driven example in Section 9.1.4, where we generate initial conditions by mimicking observations of a giant molecular cloud.
- In multi-step initialization we use a series of simulations to progressively work up to the full complexity of the multi-physics problem. An example, provided in Section 8.7.3, explains how to simulate the evolution of planetary systems in a stellar cluster. In Section 10.3.3 (see also Figure 6.10) we go a step further by exploring the hydrodynamical collapse of a giant molecular cloud that forms stars with circumstellar disks. The disks, irradiated by the neighboring stars,

subsequently collapse and form planetary systems that evolve together with the dynamical evolution of the cluster.

The last conditions discussed in this itemized list are the most complex, and often require a trial series of initial conditions and more elaborate simulations in order to execute the eventual calculation. One can hardly talk about “initial conditions” in those cases, as they represent a morphable collection of data representing the system at an earlier epoch rather than a fixed starting point in time.

In this chapter we elaborate a little on the simplest concepts of generating initial conditions. Along the sojourn through this book, you will find many other examples of the generation of starting conditions for simulating astronomical phenomena.

3.2 Creating particle distributions in space

To generate a particle distribution, we will need to access the various functionalities of the particle set. In Section 2.2.1 we discussed the fundamental characteristics of this data structure. Here we illustrate some of its important points, such as creating and storing point-mass information for a collection of particles in space.

3.2.1 Initial condition for gravitational dynamics

For large systems, initial conditions are generally not specified exactly, and we must start with a random realization of some distribution function. In self-gravitating simulations of star clusters, the [Plummer \(1911\)](#) model is often adopted as an initial density profile. In AMUSE, a particle set representing a Plummer sphere of `number_of_stars` stars can be generated with

```
particles = new_plummer_model(number_of_stars)
```

The result is a `Particles` object, which contains the identities, masses, positions and velocities of the `number_of_stars` particles generated. You could inspect the content of `particles` at this point by adding an additional line

```
print(particles)
```

This would result in something like (do absolutely try this yourself!):

key	mass	radius	vx	...	x	y	z
-	mass	length	length/time	...	length	length	length
=====	=====	=====	=====	...	=====	=====	=====
120995...	1.00e-02	0.00e+00	7.666e-01	...	1.95e-01	-2.01e-01	7.47e-02
88511...	1.00e-02	0.00e+00	1.219e-01	...	-1.65e+00	-1.55e+00	-2.92e+00
19874...	1.00e-02	0.00e+00	-2.131e-01	...	3.08e-02	1.20e+00	-1.18e+00
...	...	...	...	...	...	...	...
18055...	1.00e-02	0.00e+00	-3.412e-01	...	-1.90e+00	2.35e-01	2.04e+00
46733...	1.00e-02	0.00e+00	6.214e-01	...	-2.43e-01	-1.97e-02	-1.61e-01
67731...	1.00e-02	0.00e+00	-1.089e-02	...	-8.33e-01	7.62e-01	2.43e+00
=====	=====	=====	=====	...	=====	=====	=====

Note that the output format from your code will look somewhat different, as we have abbreviated the output due to page limitations (and we did not specify the seed in the random number generator).

As noted in §2.2.1, the `key` is a unique identifier for each particle, followed by the mass, radius, velocity, and position. Except for the first (`key`) entry, these lines may appear in any order, and the columns also may appear in a different order. The first line identifies the parameter, while the second gives the unit. In this example the units are generic (scaleless) because we have not specified any units explicitly.

The Plummer model represents a self-consistent equilibrium solution to Poisson’s equation, and so is a valid description of an N -body system. But aside from the fact that it is simple and has no free dimensionless parameters in its description, it has no special claim to fame as a cluster model. Nevertheless, it is a very common choice.

```

8 import matplotlib.pyplot as plt
9 from amuse.units.nbody_system import length
10 from amuse.ic.plummer import new_plummer_model
11
12
13 def plot_plummer(number_of_particles=1000):
14     fig = plt.figure(figsize=(5, 5))
15     ax = fig.add_subplot(1, 1, 1)
16     bodies = new_plummer_model(number_of_particles)
17     ax.scatter(bodies.x.value_in(length), bodies.y.value_in(length))
18     ax.set_xlim((-1, 1))
19     ax.set_ylim((-1, 1))
20     ax.set_xlabel("X")
21     ax.set_ylabel("Y")
22     plt.savefig("plummer.pdf")
23     print("Saved figure in file plummer.pdf")

```

Code example 3.1: Routine to plot a Plummer sphere. Code fragment from `amuse.examples.plot_plummer`.

Some initial condition generators are not considered standard, and require a community code to be built/installed before usage. For example, for “fractal” initial conditions (Goodwin & Whitworth, 2004), we must have the `amuse.community.fractalcluster` package available. If this is not the case, using the `new_fractal_cluster_model` function will result in an error:

```
ModuleNotFoundError: No module named 'amuse.community.fractalcluster'
```

Figure 3.1 presents a small collection of various initial realizations common in N -body simulations: the Plummer sphere, a King model (with structure parameter $W_0 = 9$; King, 1966), the top view of a galactic disk plus bulge model (for which we adopted `Halogen` with parameters $\alpha = 1$, $\beta = 5$, $\gamma = 0.5$; Zemp *et al.*, 2008; Zemp, 2014), and a fractal distribution (with fractal dimension 1.6; Goodwin & Whitworth, 2004).

The various models are generated with calls similar to those discussed in Section 3.2. For the King model we write:

```
particles = new_king_model(N, W0=9.0)
```

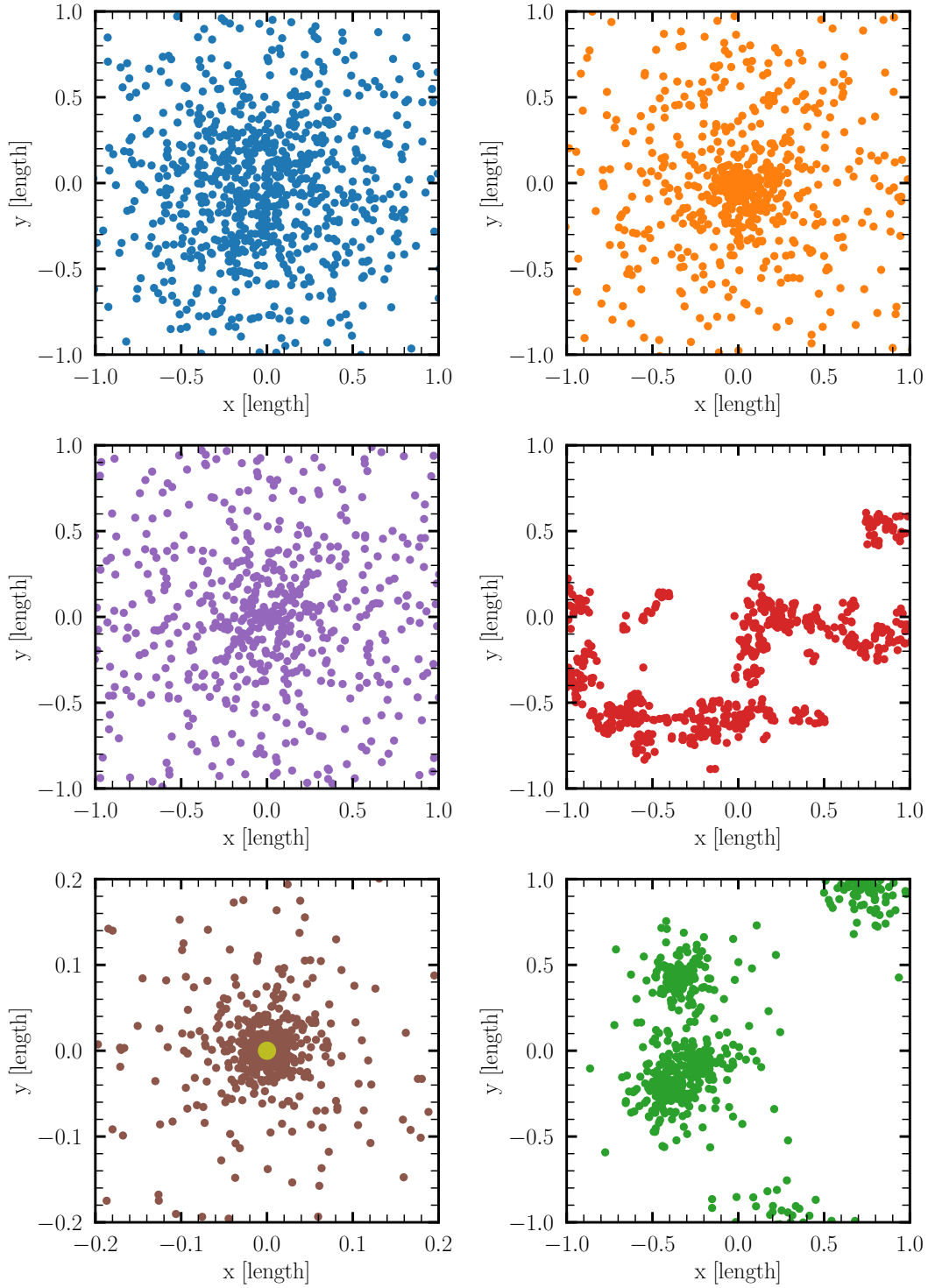


Figure 3.1: The inner parts, spanning (1 by 1 Hénon units ([Hénon, 1971](#); [Heggie & Mathieu, 1986](#))), of projected distributions drawn from Plummer model (top left), a King model with $W_0 = 9$ (top right), Halogen Galaxy model with $\alpha = 1$, $\beta = 5$, $\gamma = 0.5$ (middle left), a cluster with fractal dimension 1.6 (middle right), an Oort cloud with a central star (bottom left), and a hierarchical King model, each for $N = 1000$ particles. The source for making the Plummer distribution is presented in Code example 3.1. The script to generate this figure can be found in `amuse.examples.plot_density_distributions`.

The fractal distribution is generated with

```
particles = new_fractal_cluster_model(N=N, fractal_dimension=1.6)
```

For the fractal model (and for other special initial condition generators) one must include the appropriate file, which in this case can be loaded with

```
from amuse.ic.fractalcluster import new_fractal_cluster_model
```

These models have typically one free parameter. For the King model this is the dimensionless depth of the potential (W_0), while for the fractal model it is the fractal dimension (F_d).

A Galaxy disk model can be constructed with

```
particles = new_halogen_model(N, alpha=1.0, beta=5.0, gamma=0.5)
```

which has three free parameters (see Eq.1 in [Zemp et al., 2008](#), where these parameters are explained), A circumstellar disk can be generated using

```
from amuse.ext.protodisk import ProtoPlanetaryDisk
model = ProtoPlanetaryDisk(
    N,
    densitypower=1.5, Rmin=0.1, Rmax=1,
    q_out=1.0, discfraction=0.1,
).result
model.rotate(0., np.pi/6, np.pi/6)
```

This function has 10 arguments but the most important include the number of particles (N), the slope of the power-law projected density profile, the inner and outer disk radius (if defined in units these values are using the converter, see Section 2.3.1), the Toomre Q -parameter, and the mass fraction (here called **diskfraction**, indicating that the disk is 10% of the star’s mass).

The best way to see what parameters a certain function in AMUSE has is by reading the actual routine. In this example, the routine lives in the `amuse.ext.protodisk` module. Every code you load has an associated source code, which you can read or copy. For this you will have to dive into the directory structure. Best is probably to take some time to make yourself familiar with AMUSE’s directory structure.

The routine `ProtoPlanetaryDisk` returns a class whose parameter **result** is a particle set with additional information for performing hydrodynamical simulations, such as the internal energy, which can be used to calculate the temperature and pressure. A newly generated disk will be oriented in the x - y plane. We can rotate the disk, in azimuth (or yaw by rotation around the vertical z -axis), and elevation (or pitch, by rotation around the horizontal y axis). Here we adopted an angle of $\pi/6$ radians for both rotation angles. The result is presented in the bottom left panel in Figure 3.1.

To illustrate how we can make somewhat more complex initial conditions, we generate a composite model by first making a King model for the center of masses of the stars, and subsequently make as many King models as center of masses were generated. Here is an example of 5 King models with one-fifth the total number of stars each, as presented in the bottom right panel in Figure 3.1.

```

model_com = new_king_model(5, W)
for i in range(5):
    model = new_king_model(int(number_of_stars / 5), W)
    # Shrink the cluster.
    model.position *= 0.1
    # Keep virialized.
    model.velocity *= 3.2
    # Move to the proper coordinates.
    model.position += model_com[i].position
    model.velocity += model_com[i].velocity

```

3.2.2 Initial condition for hydrodynamics

Like particle distributions for gravitational dynamical simulations, hydrodynamical simulations also require input density and other distributions. AMUSE provides most support for smoothed particle hydrodynamics (SPH) codes (see Chapter 6), and contains a variety of distribution-generating functions. Code example 3.2 illustrates the conversion of a 1-dimensional stellar evolution model to a 3-dimensional hydrodynamical particle distribution.

```

21 def stellar_model(
22     number_of_particles=10000, mass_star=2.0 | units.MSun, time=0.0 | units.Myr
23 ):
24     star = Particle(mass=mass_star)
25     stellar_evolution = Evtwin()
26     se_star = stellar_evolution.particles.add_particle(star)
27     print(f"Evolving {star.mass} to t= {time.in_(units.Myr)}")
28     stellar_evolution.evolve_model(time)
29     print(f"Stellar type: {se_star.stellar_type}")
30     print("Creating SPH particles from the (1D) stellar evolution model")
31     sph_particles = convert_stellar_model_to_sph(
32         se_star, number_of_particles
33     ).gas_particles
34     stellar_evolution.stop()
35     return sph_particles

```

Code example 3.2: Converting a 1-D stellar evolution model to a 3-D SPH particle representation. Code fragment from `amuse.examples.plot_hydro_density_distributions`.

In Code example 3.2, in lines 25-27 the parameter `gas_particles` is returned from the function `convert_stellar_model_to_sph`. This is common practice when the called function returns a class, in this case a particle set. The return values are stored in a class containing the particle set `core_particle`, its radius (`core_radius`), and a `gas_particle` set. The resulting hydrodynamical models are plotted in Figure 3.2 (left panel); Code example 3.3 shows a portion of the plotting code.

We don't have to start from a stellar model, of course. Code example 3.4 illustrates the creation of a 3-dimensional hydrodynamical particle distribution describing a turbulent molecular cloud. As discussed in Section 3.2.1 in relation to the function `ProtoPlanetaryDisk()`, `molecular_cloud()` also returns a class whose parameter result contains the SPH particle set:

```

42 def plot_zams_stellar_model(number_of_particles=10000, mass_star=2.0 | units.MSun):
43     length_unit = units.RSun
44     sph_particles = stellar_model(number_of_particles, mass_star)
45     figure = plt.figure(figsize=(6, 6))
46     ax = figure.add_subplot(1, 1, 1)
47     sph_particles_plot(
48         sph_particles,
49         min_size=500,
50         max_size=500,
51         alpha=0.01,
52         view=(-2, 2, -2, 2) | length_unit,
53     )
54     ax.set_facecolor("white")
55     ax.set_xlabel(f"x [{length_unit}]")
56     ax.set_ylabel(f"y [{length_unit}]")
57
58     save_file = "stellar_2MSunZAMS_projected.pdf"
59     plt.savefig(save_file)
60     print(f"Saved figure in file {save_file}\n")

```

Code example 3.3: Plotting the 3-D particle representation using the AMUSE plot module. Code fragment from `amuse.examples.plot_hydro_density_distributions`.

```

67 def gmc_model(
68     number_of_particles=10000,
69     mass_cloud=10000.0 | units.MSun,
70     radius_cloud=10.0 | units.parsec,
71 ):
72     converter = nbody_system.nbody_to_si(mass_cloud, radius_cloud)
73     sph_particles = molecular_cloud(
74         target_number_of_particles=number_of_particles,
75         convert_nbody=converter,
76     ).result
77     sph = Fi(converter)
78     sph.gas_particles.add_particles(sph_particles)
79     sph.evolve_model(1 | units.day)
80     channel = sph.gas_particles.new_channel_to(sph_particles)
81     channel.copy()
82     sph.stop()
83     return sph_particles

```

Code example 3.4: Converting a molecular cloud model to a 3-D SPH particle representation. Code fragment from `amuse.examples.plot_hydro_density_distributions`.

```

from amuse.ic.molecular_cloud import new_molecular_cloud

sph_particles = new_molecular_cloud(
    target_number_of_particles=number_of_particles, convert_nbody=converter,
)

```

Both routines can use `convert_nbody` converter to scale between dimensionless N-body units and physical units. The result is a molecular cloud represented by N particles. The particles will have all have the same mass (`mass`) and internal energy (`u`), and positions and velocities consistent with a homogeneous gas sphere with a turbulent ve-

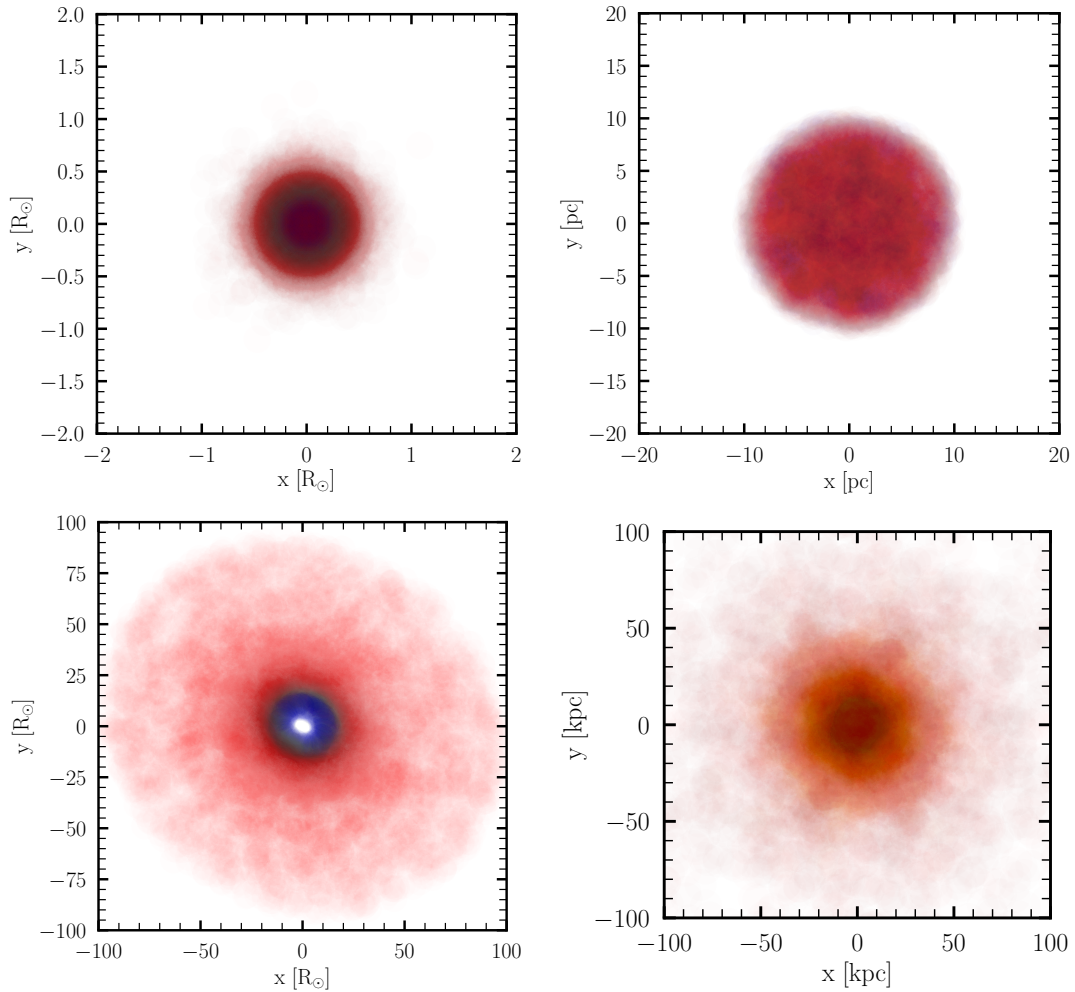


Figure 3.2: Some initial particle distributions for SPH codes. The top-row left panel shows a 2 solar mass star, the right a homogeneous spherical distribution with a turbulent velocity field, as is often used in simulations of molecular clouds. Particles are colored by their internal energy, from red (0.048 (km/s)^2) to blue (1.82 (km/s)^2). The bottom row shows a slightly tilted circumstellar disk with a total mass of 0.1% of the star’s mass (left), and a $10^{10} M_{\odot}$ galaxy model. The script to generate this figure can be found in `amuse.examples.plot_hydro_density_distributions`.

locity field. (The system is integrated for a short time after creation to reduce random initial noise in the gas distribution.) The resulting density distribution is presented in Figure 3.2 (right panel). We will explore such models in more depth in Chapter 6.

We will not dwell here on more advanced code-coupling strategies, and defer discussion of radiative transport to Chapter . We postpone these advanced topics because they require multiple codes to be operational at the same time. Such coupling is discussed toward the end of this book, in Chapters 7 and 9.

3.3 Creating mass distributions

The default for `new_plummer_model` and `new_fractal_cluster_model` is to create stars of equal mass, but usually we want stars to have a range of masses, drawn from

some distribution. This distribution is known as the initial mass function, and is one of the most important quantities in stellar astrophysics.

The initial mass function is often represented as a power-law, meaning that the number of stars dN with masses between m and $m + dm$ is given by

$$\frac{dN}{dm} = A m^\alpha \quad (3.1)$$

for $M_{\min} < m < M_{\max}$. The choice $\alpha = -2.35$ (Salpeter, 1955) is so common that it is has its own standard function call in AMUSE:

```
mass_zams = new_salpeter_mass_distribution(
    number_of_stars, mass_min=mass_min, mass_max=mass_max
)
```

returns an array of `number_of_stars` masses randomly selected between `mass_min` and `mass_max`, distributed according to a Salpeter distribution, a power-law distribution with exponent $\alpha = -2.35$. The resulting mass function is presented in Figure 3.3. The naming of this mass `mass_zams` seems somewhat arcane, but refers to the mass of a star at the moment of hydrogen ignition (generally called the birth of the star) which is referred to as the zero-age main-sequence (see also Section 5.1).

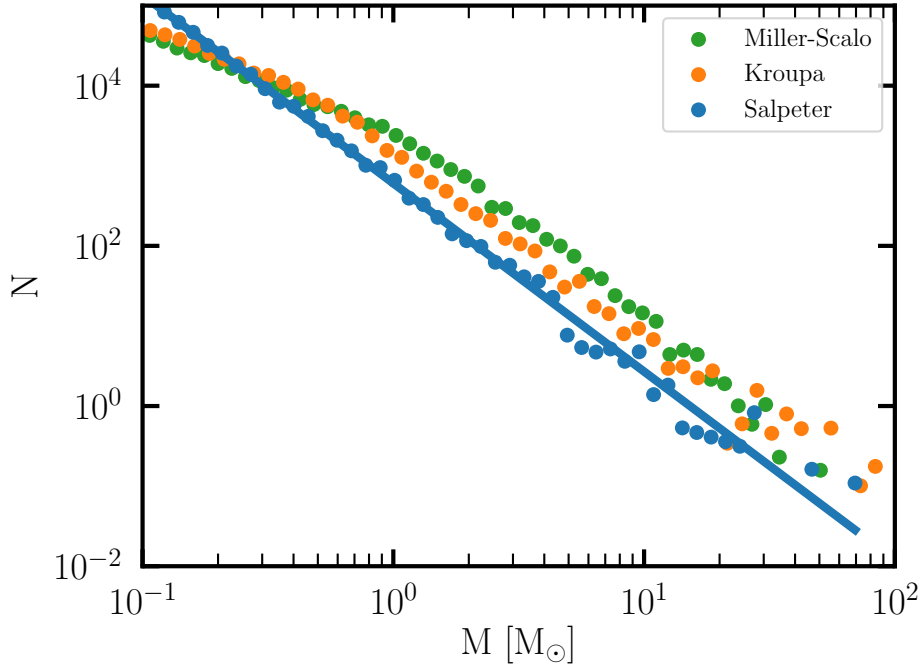


Figure 3.3: Initial mass function for 10^4 stars between $0.1 M_\odot$ and $100 M_\odot$. The Salpeter mass function is overplotted with an analytic power-law with a slope of -2.35 (blue). Two other often used mass functions are shown in green and orange. The latter two are described by Miller & Scalo (1979) (in green) and Kroupa (2001) (orange). An example of how this is plotted is presented in Code example 3.5. The script to generate this figure can be found in `amuse.examples.salpeter`.

```

18 def plot_mass_function(masses, ximf, color, label, n_bins=51):
19     "Plots a histogram of the masses provided, with a power law added if ximf > 0"
20     mass_min = masses.min()
21     mass_max = masses.max()
22     log_min = np.log10(mass_min.value_in(units.MSun))
23     log_max = np.log10(mass_max.value_in(units.MSun))
24     bins = np.logspace(log_min, log_max, n_bins)
25     print(bins)
26     bin_number, bin_edges = np.histogram(masses.value_in(units.MSun), bins=bins)
27     y = bin_number / (bin_edges[1:] - bin_edges[:-1])
28     x = (bin_edges[1:] + bin_edges[:-1]) / 2.0
29     for yi in y:
30         yi = max(yi, 1.0e-10)
31
32     plt.scatter(x, y, s=50, c=color, lw=0, label=label)
33
34     if ximf < 0:
35         c = (
36             (mass_max.value_in(units.MSun) ** (ximf + 1))
37             - (mass_min.value_in(units.MSun) ** (ximf + 1))
38         ) / (ximf + 1)
39         plt.plot(x, len(masses) / c * (x**ximf), c=color)

```

Code example 3.5: Routine to plot the generated Salpeter mass function. Code fragment from `amuse.examples.salpeter`.

If you need a power-law mass function with a different slope, you can change the exponent by using a keyword argument, but we recommend that you use the more general version of the mass function generator:

```

mass_zams = new_powerlaw_mass_distribution(
    1000,
    mass_min=0.1 | units.MSun,
    mass_max=100 | units.MSun,
    alpha=-2.0,
)

```

which will return a list of 1000 stars with masses between 0.1 and 100 $M_{\odot}$, distributed according to a power law with $\alpha = -2$.

The masses can now be easily assigned to the previously generated Plummer distribution, by

```
particles.mass = mass_zams
```

or one can generate a new particle set:

```
particles = Particles(mass=mass_zams)
```

The particles can subsequently be assigned additional attributes, as already discussed. Note that changing the masses causes the system to have a different total mass and a different ratio of potential to kinetic energy, as anticipated in the function used to generate the positions and velocities, and the system generally needs to be re-scaled (see Section 4.3.1).

There are several other popular mass functions for which we generated special routines, such as [Miller & Scalo \(1979\)](#)

```
particles.mass = new_miller_scalo_mass_distribution()
```

and [Kroupa \(2001\)](#):

```
particles.mass = new_kroupa_mass_distribution()
```

A generalization of these broken power-law mass distributions is available in the class `MultiplePartIMF`, which takes arrays of breakpoints and power-law slopes allowing the user to construct a mass function of personal choice.

3.4 Creating systems of multiple stars and planets

3.4.1 The Solar system

The initial conditions of the solar system are unknown. But if we are interested in the future dynamics of the solar system we could start with the masses, positions, and velocities of the Sun and eight planets as they are observed at some moment in time, and then integrate the equations of motion forward in time. Table 3.1 presents such conditions from which such a simulation can be started.

name	mass [M_{Jup}]	position [au]			velocity [km/s]		
Sun	1047.517	0.005717	-0.00538	-2.130×10^{-5}	0.007893	0.01189	0.0002064
Mercury	0.000174	-0.31419	0.14376	0.035135	-30.729	-41.93	-2.659
Venus	0.002564	-0.3767	0.60159	0.03930	-29.7725	-18.849	0.795
Earth	0.003185	-0.98561	0.0762	-7.847×10^{-5}	-2.927	-29.803	-0.000533
Mars	0.000338	-1.2895	-0.9199	-0.048494	14.900	-17.721	0.2979
Jupiter	1.000000	-4.9829	2.062	-0.10990	-5.158	-11.454	-0.13558
Saturn	0.299470	-2.075	8.7812	0.3273	-9.9109	-2.236	-0.2398
Uranus	0.045737	-12.0872	-14.1917	0.184214	5.1377	-4.7387	-0.06108
Neptune	0.053962	3.1652	29.54882	0.476391	-5.443317	0.61054	-0.144172

Table 3.1: A realization of the Solar System, up to the last known significant digit, calculated for 5 April 2063 at 00:00 hours—Julian date 2474649.5 days (corresponding to first contact: <http://www.startrek.com/article/origin-of-first-contact-day-explained>)—using the ephemerides from http://ssd.jpl.nasa.gov/txt/p_elem_t2.txt, estimating the uncertainty in the positions and velocities using the dispersion given by [Folkner \(2011\)](#).

The orbital parameters from this table are coded in the function `new_solar_system` (in the `amuse.ic.solarsystem` module). The parameters for moons, planetesimals, Trojans, Greeks, plutinos, the classic Kuiper belt, and the Oort cloud are not included in this table. For the moons we have a separate function that generates initial conditions for all the Solar system’s moons, it is called `new_lunar_system_in_time` in the `amuse.ic.solar_system_moons` module. For the asteroids and other minor bodies, you will have to download the appropriate data, and convert the ephemerides to Cartesian coordinates using your own routine (see the assignment in Section 3.6.2).

```

12 from amuse.datamodel import Particles
13 from amuse.units import units
14
15
16 def new_sun_venus_and_earth():
17     particles = Particles(3)
18     sun = particles[0]
19     sun.name = "Sun"
20     sun.mass = 1.0 | units.MSun
21     sun.radius = 1.0 | units.RSun
22     sun.position = (855251, -804836, -3186) | units.km
23     sun.velocity = (7.893, 11.894, 0.20642) | (units.m / units.s)
24
25     venus = particles[1]
26     venus.name = "Venus"
27     venus.mass = 0.0025642 | units.MJupiter
28     venus.radius = 3026.0 | units.km
29     venus.position = (-0.3767, 0.60159, 0.03930) | units.au
30     venus.velocity = (-29.7725, -18.849, 0.795) | units.kms
31
32     earth = particles[2]
33     earth.name = "Earth"
34     earth.mass = 1.0 | units.MEarth
35     earth.radius = 1.0 | units.REarth
36     earth.position = (-0.98561, 0.0762, -7.847e-5) | units.au
37     earth.velocity = (-2.927, -29.803, -0.0005327) | units.kms
38
39     sun.color = "#FFFF00"
40     venus.color = "wheat"
41     earth.color = "deepskyblue"
42
43     particles.move_to_center()
44     return particles

```

Code example 3.6: Code to generate initial conditions for Venus and Earth orbiting the Sun. Code fragment from `amuse.examples.sun_venus_earth`.

The source code to initialize the Sun, Venus, and Earth is presented in Code example 3.6. It starts by including two basic AMUSE functionalities—particles and units:

```

10 from amuse.datamodel import Particles
11 from amuse.units import units

```

A particle set is then declared in line 14:

```

14 particles = Particles(3)

```

Attribute values are then set. The first particle in the array (`particle[0]`) is assigned to the Sun and the other two particles to Venus and Earth. (Note the details of setting three vector coordinates and units in a single line.) Then, writing

```

print(sun.position)

```

will print out the Cartesian coordinates of the Sun. After initialization, the particles are moved to the center of mass of the 3-body system, using the (class-defined) function `move_to_center()` mentioned in the previous subsection. At the end of the function, the particle set is returned to the main script with

```
34     return particles
```

A potentially easier way to achieve the initialization of the Sun, Venus and Earth, would be to use the following few lines:

```
from amuse.ic.solarsystem import new_solar_system
solar_system = new_solar_system()
SunVenusEarth = solar_system[0] + solar_system[2] + solar_system[3]
SunVenusEarth.move_to_center()
```

The third particle in this array actually represents the center of mass of the Earth-Moon system. To separate these two objects, one could use this (somewhat verbose and slow, but more precise) version:

```
from amuse.ic.solar_system_moons import new_lunar_system
solar_system = new_lunar_system(Julian_date=2474649.5 | units.day)
Sun = solar_system[solar_system.name=="sun"]
Venus = solar_system[solar_system.name=="Venus"]
Earth = solar_system[solar_system.name=="Earth"]
SunVenusEarth = Sun + Venus + Earth
SunVenusEarth.move_to_center()
```

In this latter script one actually uses the mass, position and velocity of Earth, rather than of the Earth-Moon system at Julian date 2474649.5.

The integration of the orbits can be carried out using the script in Code example 4.2 (see Section 4.4.1). Code example 3.7 presents a snippet of code to plot the planets' positions as they orbit the Sun. The results of the calculation are shown in Figure 3.4. We use `matplotlib.pyplot` (imported via convention as `plt`) to present the results, and later will show how to use an AMUSE-tailored version for plotting with units and hydrodynamical models. Note that here and throughout we provide references to the scripts that produced the figures and, for convenience, we save the plot in a file (`plt.savefig(...)`) in the current directory, and also display it on the screen (`plt.show()`).

Here we probably should explain the two function calls on lines 97 and 99, which return arrays of the requested variables (here the Cartesian coordinates along the *x* and *y* axes) for the requested particle in astronomical units. The basic functioning of this routine, as a short stand-alone snippet, might look as follows:

```
time_series = read_set_from_file("stored_particle_time_series.amuse")
bodies = time_series.history[0]
Earth = bodies[bodies.name == "Earth"]
x = get_timeline_of_attribute_as_vector(time_series, Earth, "x")
```

We do explain this in more detail in Appendix A, though.

```

80 def plot_track(particles, output_filename):
81     figure = plt.figure(figsize=(10, 10))
82     plt.rcParams.update({"font.size": 30})
83     ax = figure.add_subplot(1, 1, 1)
84     ax.minorticks_on()
85     ax.locator_params(nbins=3)
86
87     x_label = "x [au]"
88     y_label = "y [au]"
89     plt.xlabel(x_label)
90     plt.ylabel(y_label)
91
92     ax.scatter([0.0], [0.0], color="#CCCC00", lw=8)
93     for planet in particles:
94         if planet.name.lower() in ["venus", "earth"]:
95             ax.plot(
96                 planet.get_timeline_of_attribute_as_vector("x")[1].value_in(units.au),
97                 planet.get_timeline_of_attribute_as_vector("y")[1].value_in(units.au),
98                 color=planet.color,
99             )
100     ax.set_xlim(-1.3, 1.3)
101     ax.set_ylim(-1.3, 1.3)
102
103     plt.savefig(f"{output_filename}.pdf")
104     print(f"\nSaved figure in file {output_filename}\n")

```

Code example 3.7: Plotting the orbits of Venus and Earth around the Sun using pyplot. This tells you more about pyplot than about AMUSE, but we present it here for completeness. Code fragment from `amuse.examples.sun_venus_earth`.

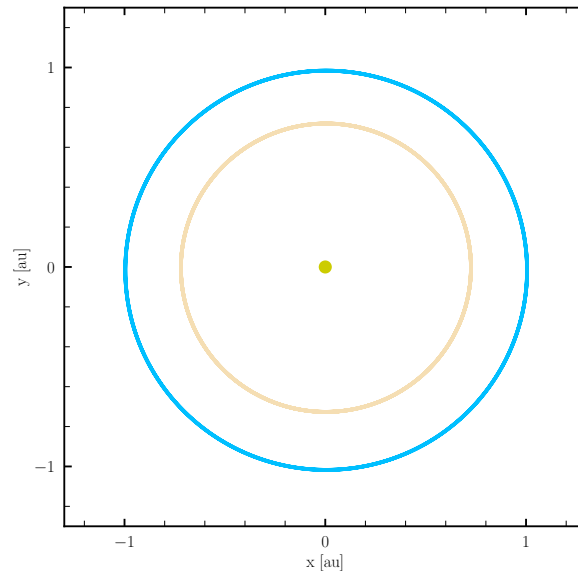


Figure 3.4: Orbital integration of the Sun, Venus and Earth for 1 yr using the script in Code example 4.2. The script to generate this figure can be found in `amuse.examples.sun_venus_earth`.

3.4.2 Other planetary systems

In principle, any planetary system can be initialized in the above described method, by providing masses, positions and velocities of the planets. Following planet formation through Oligarchic growth ([Kokubo & Ida, 2002](#))

```
from amuse.ic import make_planets_oligarch
from amuse.units import units
mass_star = 1 | units.MSun
radius_star = 1 | units.RSun
inner_radius_disk = 10 * radius_star
outer_radius_disk = 42 | units.au
mass_disk = 0.01 * mass_star
planetary_system = make_planets_oligarch.new_system(
    mass_star,
    radius_star,
    inner_radius_disk,
    outer_radius_disk,
    mass_disk,
)
print(planetary_system.planets)
```

and to print the planetary data

```
print(planetary_system.planets[0])
```

Integrating such a system can be achieved by

```
gravity = Ph4()
gravity.particles.add_particles(planetary_system)
gravity.particles.add_particles(planetary_system.planets[0])
```

The implementation of the oligarchic planet formation model was designed by [Tremaine \(2015\)](#) and we adopt the implementation by [Hansen & Murray \(2013\)](#).

```
from amuse.ic import make_planets_oligarch
mass_star = 1 | units.MSun
radius_star = 1 | units.RSun
inner_radius_disk = 10 * radius_star
outer_radius_disk = 42 | units.au
mass_disk = 0.01*mass_star
planetary_system = make_planets_oligarch.new_system(
    mass_star,
    radius_star,
    inner_radius_disk,
    outer_radius_disk,
    mass_disk,
)
```

3.4.3 Binaries and hierarchical multiples

Generating binaries and hierarchical multiple systems may be somewhat elaborate, but if done systematically it is not hard. It is best to start with the innermost (hardest)

binary (or more than one if the system is highly hierarchical).

Let's start by making a simple binary, as in Code example 3.8. This might be a planet with moon, or a star with planet, or a galaxy with a dwarf companion. We assume that both objects are point masses, and that they have a Keplerian orbit.

```

35 def new_binary_orbit(stars, kepler):
36     """
37     Takes a Particles object of length 2 and returns a binary particle from the
38     stars, using the values from `kepler`
39     """
40     if len(stars) != 2:
41         raise ValueError("'stars' must consist of exactly two particles")
42     rel_position = as_vector_quantity(kepler.get_separation_vector())
43     rel_velocity = as_vector_quantity(kepler.get_velocity_vector())
44
45     mu = stars[0].mass / stars.mass.sum()
46     binary = Particle()
47     binary.child1 = stars[0]
48     binary.child2 = stars[1]
49     binary.child1.position = mu * rel_position
50     binary.child2.position = -(1 - mu) * rel_position
51     binary.child1.velocity = -(1 - mu) * rel_velocity
52     binary.child2.velocity = mu * rel_velocity
53     print(binary.child1.position.in_(units.au))
54     print(binary.child2.position.in_(units.au))
55     return binary

```

Code example 3.8: Source for generating a binary of two objects with masses, positions and velocities of the two stars `star[0].mass` and `star[1].mass`. Other orbital parameters can subsequently be calculated from a Kepler solver, as initialized in Code example 3.9. Source code is available in `amuse.examples.make_binary_star`.

The Kepler solver in this routine is presented in Code example 3.9.

```

22 def new_kepler(converter):
23     """Starts and returns a new Kepler integrator with given converter"""
24     kepler = Kepler(converter)
25     kepler.initialize_code()
26     kepler.set_longitudinal_unit_vector(1.0, 0.0, 0.0)
27     kepler.set_transverse_unit_vector(0.0, 1.0, 0)
28     return kepler

```

Code example 3.9: Initializing a Kepler object for system in the x-y plane. Source code is available in `amuse.examples.make_binary_star`.

After the binary is generated, one would like to see if it really turned out as expected. A simple way to achieve this is presented in Code example 3.10, where we calculate the semi-major axis and eccentricity from the binary's Cartesian coordinates.

A higher order hierarchical multiple can be generated by iteratively using the generated binary as one of the stars in the binary generation routines.

```

62 def calculate_orbital_elements(bi, kepler):
63     """
64     Takes a binary particle and a kepler code, returns the semi-major axis and
65     eccentricity
66     """
67     comp1 = bi.child1
68     comp2 = bi.child2
69     mass = comp1.mass + comp2.mass
70     pos = comp2.position - comp1.position
71     vel = comp2.velocity - comp1.velocity
72     kepler.initialize_from_dyn(
73         mass,
74         pos[0],
75         pos[1],
76         pos[2],
77         vel[0],
78         vel[1],
79         vel[2],
80     )
81     a, e = kepler.get_elements()
82     return a, e

```

Code example 3.10: Code snippet for verifying the binary's orbital parameters. Source code is available in `amuse.examples.make_binary_star`.

3.5 Often used initial conditions

For practical purposes we have generated a series of often-used initial conditions. These are collected in a separate Python package with a number of generation functions. These functions take the form of `make_`, `add_`, `conv_` and `move_` for generating a new particle set, adding some attribute or feature, converting the particle set and moving it around. A complete list is presented in Appendix A.2.1.

These utilities can read an existing particle set from a file and write the extended or otherwise modified particle set to another file, after which it can be read in by one of the other utilities. In this way it is possible to generate rather elaborate particle sets.

For example, a Plummer distribution of 100 stars in space of which 10 receive a system of 8 planets following the Oligarchic growth model on the Sun's location in the Galaxy can be realized as follows.

```

python make_Plummer_model.py -N 100 -F plummer.amuse
python name_stars.py -f plummer.amuse --name PlanetarySystem \
    --nstars 10 -F plummerNamed.amuse
python add_planet_Oligarch.py -f PlummerNamed.amuse --name PlanetarySystem \
    --Nplanets 8 -F plummerWithPlanets.amuse
python move_all_particles.py -f plummerWithPlanets.amuse --pos -8300 0.0 27 \
    --vel 11.1 240 7.25 -F plummerWithPlanetsInGalaxy.amuse

```

This procedure results in a number of files `plummer.amuse`, `plummerNamed.amuse`, `plummerWithPlanets.amuse` and the final snapshot named `plummerWithPlanetsInGalaxy`. This latter snapshot can subsequently be run with a gravity code including a Galactic potential to study the evolution of the stellar system with planets.

By combining various routines it is possible to create a broad range of particle sets that can be used as initial conditions. Of course, not all varieties of initial conditions are available, but we expect the list of initial condition routines to grow steadily with time.

3.6 Assignments

3.6.1 Create your own initial conditions generator or manipulator

The more routines we have for generating initial conditions, the more versatile AMUSE will become. For this assignment requires some idea of how these generators and manipulators are constructed. Make a generator, constructor, manipulator for an application that is not yet available in AMUSE — for example, one to generate a gaseous circumstellar disk around a single star. By the time you read this, such a routine probably has already been added to the repository, so you should find another example for which you can make a generator. Test the code, and submit it to the git repository at <https://github.com/amusecode/initial-conditions-generator>.

3.6.2 Solar system minor bodies

Download the ephemerides of the Solar system minor bodies (<https://www.minorplanetcenter.net/iau/mpc.html>), and generate a routine to convert the Kepler elements to Cartesian coordinates.

Take the initial conditions for the solar system (Table 3.1) and integrate the planets' orbits for 100 yr with the integrator of your choice (Table 4.1). Explain why you have selected this integrator, and discuss the effect of using another integrator.

1. Generate a snapshot for the Solar system at the time of first contact.
2. Check the planet positions and velocities from Table 3.1.

Bibliography

- Folkner, W. M. 2011 (Oct.). Uncertainties in the JPL planetary ephemeris. *Pages 43–48 of: Capitaine, Nicole (ed), Journées Systèmes de Référence Spatio-temporels 2010.*
- Goodwin, S. P., & Whitworth, A. P. 2004. The dynamical evolution of fractal star clusters: The survival of substructure. *A&A*, **413**(Jan.), 929–937.
- Hansen, Brad M. S., & Murray, Norm. 2013. Testing in Situ Assembly with the Kepler Planet Candidate Sample. *ApJ*, **775**(1), 53.
- Heggie, D. C., & Mathieu, R. D. 1986. Standardised Units and Time Scales. *Page 233 of: Hut, Piet, & McMillan, Stephen L. W. (eds), The Use of Supercomputers in Stellar Dynamics*, vol. 267.

- Hénon, M. H. 1971. The Monte Carlo Method (Papers appear in the Proceedings of IAU Colloquium No. 10 Gravitational N-Body Problem (ed. by Myron Lecar), R. Reidel Publ. Co. , Dordrecht-Holland.). *Ap&SS* , **14**(1), 151–167.
- King, Ivan R. 1966. The structure of star clusters. III. Some simple dynamical models. *AJ* , **71**(Feb.), 64.
- Kokubo, Eiichiro, & Ida, Shigeru. 2002. Formation of Protoplanet Systems and Diversity of Planetary Systems. *The Astrophysical Journal*, **581**(1), 666.
- Kroupa, Pavel. 2001. On the variation of the initial mass function. *MNRAS* , **322**(2), 231–246.
- Miller, G. E., & Scalo, J. M. 1979. The initial mass function and stellar birthrate in the solar neighborhood. *ApJS* , **41**(Nov.), 513–547.
- Plummer, H. C. 1911. On the problem of distribution in globular star clusters. *MNRAS* , **71**(Mar.), 460–470.
- Salpeter, Edwin E. 1955. The Luminosity Function and Stellar Evolution. *ApJ* , **121**(Jan.), 161.
- Tremaine, Scott. 2015. The Statistical Mechanics of Planet Orbits. *The Astrophysical Journal*, **807**(2), 157.
- Zemp, Marcel. 2014 (July). *Halogen: Multimass spherical structure models for N-body simulations*. Astrophysics Source Code Library, record ascl:1407.020.
- Zemp, Marcel, Moore, Ben, Stadel, Joachim, Carollo, C. Marcella, & Madau, Piero. 2008. Multimass spherical structure models for N-body simulations. *MNRAS* , **386**(3), 1543–1556.